EDA&V ASS 01 (0018-0030)

October 19, 2025

1 Exploratory Data Analysis & Business Insights

This notebook explores the sales CRM dataset consisting of five related tables: **accounts**, **products**, **sales teams**, **sales pipeline**, and a **data dictionary**. It is structured around 40 business questions (14 basic, 14 intermediate, and 12 advanced) as outlined in the assignment instructions. Each section contains:

- A **question** phrased from a business perspective (without referring to table names).
- A Python **code cell** that computes and/or visualises the answer.
- A brief **interpretation** explaining the findings in a business context.

1.0.1 Dataset Description

The **CRM Sales Opportunities dataset** from Maven Analytics contains simulated business-to-business (B2B) sales data for a fictional company's customer relationship management system. It includes **five interconnected tables** — `accounts`, `products`, `sales_teams`, `sales_pipeline`, and a `data_dictionary`. Together, they capture company details, product offerings, sales team structures, and the entire sales funnel from initial prospecting to deal closure.

- **Accounts** holds customer organization details such as sector, revenue, employee count, and office location.
- **Products** lists items and pricing by product family and series.
- **Sales_teams** describes sales agents and their regional offices.
- **Sales_pipeline** tracks opportunities with attributes like stage, expected value, open and close dates, and linked account/product/team IDs.
- **Data_dictionary** provides metadata definitions for all fields.

The dataset spans several months of activity, enabling analysis of sales performance, win rates, forecasting, and segmentation. It is structured for relational joins across key IDs (e.g., `account`, `product`, and `sales_agent`) and is suitable for comprehensive exploratory data analysis and business insight generation.

1.0.2 Data Preparation

- Before analysis, essential columns were converted to the correct data types to ensure consistency and accurate computation.

- Numeric conversion: Revenue, employee count, sales price, and close value columns were converted to numeric values.
- Date conversion: Engage and close dates were converted to datetime format for time-based analysis.
- Quality checks: Everything else that needed handling, such as missing values, duplicates, or other specific issues, was addressed within the relevant questions where required.

The dataset is loaded once at the beginning, and samples of each table are displayed.

[1]:
```
# Limit BLAS threads for reproducible performance in notebooks
import os
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

# Core libs
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import Latex, display

# Plot style
sns.set_style("whitegrid")

#  Load data
accounts       = pd.read_csv("accounts.csv")
products       = pd.read_csv("products.csv")
sales_pipeline = pd.read_csv("sales_pipeline.csv")
sales_teams    = pd.read_csv("sales_teams.csv")
data_dict      = pd.read_csv("data_dictionary.csv")
data_prf       = pd.read_csv("dataset_profile.csv")

#  Clean/convert key fields
accounts["revenue_numeric"]   = pd.to_numeric(accounts["revenue"],␣
 ↪errors="coerce")
accounts["employees_numeric"] = pd.to_numeric(accounts["employees"],␣
 ↪errors="coerce")
products["sales_price_numeric"] = pd.to_numeric(products["sales_price"],␣
 ↪errors="coerce")
sales_pipeline["close_value_numeric"] = pd.
 ↪to_numeric(sales_pipeline["close_value"], errors="coerce")
sales_pipeline["engage_date"] = pd.to_datetime(sales_pipeline["engage_date"],␣
 ↪errors="coerce")
sales_pipeline["close_date"]  = pd.to_datetime(sales_pipeline["close_date"], ␣
 ↪errors="coerce")
```

```python
#  Quick previews
print("accounts.csv");        display(accounts.head())
print("products.csv");        display(products.head())
print("sales_pipeline.csv");  display(sales_pipeline.head())
print("sales_teams.csv");     display(sales_teams.head())
print("data_dictionary.csv"); display(data_dict.head())

print("dataset_profile.csv for dataset initial understanding")
display(data_prf)
```

```
accounts.csv

            account     sector  year_established  revenue  employees  \
0  Acme Corporation  technolgy              1996  1100.04       2822
1        Betasoloin    medical              1999   251.41        495
2          Betatech    medical              1986   647.18       1185
3        Bioholding    medical              2012   587.34       1356
4           Bioplex    medical              1991   326.82       1016

  office_location subsidiary_of  revenue_numeric  employees_numeric
0   United States           NaN          1100.04               2822
1   United States           NaN           251.41                495
2           Kenya           NaN           647.18               1185
3      Philipines           NaN           587.34               1356
4   United States           NaN           326.82               1016

products.csv

        product series  sales_price  sales_price_numeric
0     GTX Basic    GTX          550                  550
1       GTX Pro    GTX         4821                 4821
2    MG Special     MG           55                   55
3   MG Advanced     MG         3393                 3393
4  GTX Plus Pro    GTX         5482                 5482

sales_pipeline.csv

  opportunity_id      sales_agent         product  account deal_stage  \
0       1C1I7A6R      Moses Frase  GTX Plus Basic  Cancity        Won
1       Z063OYW0  Darcel Schlecht          GTXPro    Isdom        Won
2       EC4QE1BX  Darcel Schlecht      MG Special  Cancity        Won
3       MV1LWRNH      Moses Frase       GTX Basic  Codehow        Won
4       PE84CX4O        Zane Levy       GTX Basic   Hatfan        Won

  engage_date close_date  close_value  close_value_numeric
0  2016-10-20 2017-03-01       1054.0               1054.0
1  2016-10-25 2017-03-11       4514.0               4514.0
2  2016-10-25 2017-03-07         50.0                 50.0
3  2016-10-25 2017-03-09        588.0                588.0
4  2016-10-25 2017-03-02        517.0                517.0
```

sales_teams.csv

	sales_agent	manager	regional_office
0	Anna Snelling	Dustin Brinkmann	Central
1	Cecily Lampkin	Dustin Brinkmann	Central
2	Versie Hillebrand	Dustin Brinkmann	Central
3	Lajuana Vencill	Dustin Brinkmann	Central
4	Moses Frase	Dustin Brinkmann	Central

data_dictionary.csv

	Table	Field	Description
0	accounts	account	Company name
1	accounts	sector	Industry
2	accounts	year_established	Year Established
3	accounts	revenue	Annual revenue (in millions of USD)
4	accounts	employees	Number of employees

dataset_profile.csv for dataset initial understanding

	table	column	dtype	non_null	nulls	unique	\
0	accounts	account	object	85	0	85	
1	accounts	sector	object	85	0	10	
2	accounts	year_established	int64	85	0	35	
3	accounts	revenue	float64	85	0	85	
4	accounts	employees	int64	85	0	85	
5	accounts	office_location	object	85	0	15	
6	accounts	subsidiary_of	object	15	70	7	
7	sales_teams	sales_agent	object	35	0	35	
8	sales_teams	manager	object	35	0	6	
9	sales_teams	regional_office	object	35	0	3	
10	products	product	object	7	0	7	
11	products	series	object	7	0	3	
12	products	sales_price	int64	7	0	7	
13	sales_pipeline	opportunity_id	object	8800	0	8800	
14	sales_pipeline	sales_agent	object	8800	0	30	
15	sales_pipeline	product	object	8800	0	7	
16	sales_pipeline	account	object	7375	1425	85	
17	sales_pipeline	deal_stage	object	8800	0	4	
18	sales_pipeline	engage_date	object	8300	500	421	
19	sales_pipeline	close_date	object	6711	2089	306	
20	sales_pipeline	close_value	float64	6711	2089	2051	
21	data_dictionary	Table	object	21	0	4	
22	data_dictionary	Field	object	21	0	18	
23	data_dictionary	Description	object	21	0	19	

	example_values
0	Acme Corporation, Betasoloin, Betatech
1	technolgy, medical, retail
2	1996, 1999, 1986

```
3                          1100.04, 251.41, 647.18
4                                  2822, 495, 1185
5                  United States, Kenya, Philipines
6      Acme Corporation, Massive Dynamic, Bubba Gump
7   Anna Snelling, Cecily Lampkin, Versie Hillebrand
8        Dustin Brinkmann, Melvin Marxen, Cara Losch
9                                Central, East, West
10                   GTX Basic, GTX Pro, MG Special
11                                     GTX, MG, GTK
12                                   550, 4821, 55
13                      1C1I7A6R, Z063OYW0, EC4QE1BX
14           Moses Frase, Darcel Schlecht, Zane Levy
15                GTX Plus Basic, GTXPro, MG Special
16                          Cancity, Isdom, Codehow
17                              Won, Engaging, Lost
18                2016-10-20, 2016-10-25, 2016-10-29
19                2017-03-01, 2017-03-11, 2017-03-07
20                              1054.0, 4514.0, 50.0
21                   accounts, products, sales_teams
22                 account, sector, year_established
23          Company name, Industry, Year Established
```

The following code imports the necessary libraries, loads all five tables, and previews sample records from each dataset.

1.0.3 Basic Question 1

How many customer companies are recorded in the data?

[2]:
```
# Count distinct customer companies
num_accounts = len(accounts)
print("Total number of customer companies:", num_accounts)
```

```
Total number of customer companies: 85
```

The dataset lists **85** distinct client companies. This shows your sales team is currently targeting a manageable portfolio rather than thousands of small accounts. Growth plans can focus on expanding this base or deepening existing relationships.

1.0.4 Basic Question 2

How many individual sales representatives does the company employ?

[3]:
```
# Count unique sales representatives
num_agents = sales_teams['sales_agent'].nunique()
print("Number of sales representatives:", num_agents)
```

```
Number of sales representatives: 35
```

There are **35** sales representatives. This moderately sized force needs to be allocated wisely across industries and regions to maximise coverage without over-stretching anyone.

1.0.5 Basic Question 3

How many active opportunities are currently in the prospecting stage?

[4]:
```
# Count deals currently in the 'Prospecting' stage
prospecting_count = (sales_pipeline['deal_stage'] == 'Prospecting').sum()
print("Prospecting opportunities:", prospecting_count)
```

```
Prospecting opportunities: 500
```

There are **500** deals in the prospecting stage, indicating a healthy pipeline of leads. Converting even a modest portion into engagements could significantly boost future revenue.

1.0.6 Basic Question 4

Which product family contains the most distinct products?

[5]:
```
# Frequency of products by series
series_counts = products["series"].value_counts()
display(series_counts.to_frame(name="count"))

# Series with the most products
max_series = series_counts.idxmax()
print("Series with most products:", max_series)
```

```
        count
series
GTX         4
MG          2
GTK         1

Series with most products: GTX
```

The **GTX** series has four variants, while MG has two and GTK has one. The broader GTX range gives customers more options and may explain why GTX products feature heavily in the pipeline.

1.0.7 Basic Question 5

What are the lowest and highest annual revenues reported across all customer companies?

[6]:
```
# Min/max client revenue (report in M and B for readability)
min_rev = accounts["revenue_numeric"].min()
max_rev = accounts["revenue_numeric"].max()

print(f"Lowest Revenue: ${min_rev:,.2f}M")
print(f"Highest Revenue: ${max_rev/1000:,.2f}B")
```

```
Lowest Revenue: $4.54M
Highest Revenue: $11.70B
```

The smallest client generates about 4.54 million in annual revenue, while the largest reaches 11.7 billion. This wide gap shows that our customer base includes both small businesses and major corporations, giving a balanced mix of stability and growth potential.

1.0.8 Basic Question 6

What is the average suggested retail price of the products offered?

[7]:
```
# Calculate and print the average product price
avg_price = products["sales_price_numeric"].mean()
print("Average product price:", avg_price)
```

```
Average product price: 6023.571428571428
```

The average list price is around **$6,023**, suggesting typical deals are in the low-thousands range. This price point places the products in a mid-market rather than premium segment.

1.0.9 Basic Question 7

How many total sales opportunities have been closed, whether won or lost?

[8]:
```
# Count opportunities in closed stages (Won + Lost)
closed_count = sales_pipeline[sales_pipeline["deal_stage"].isin(["Won",␣
↪"Lost"])].shape[0]
print("Closed opportunities:", closed_count)
```

```
Closed opportunities: 6711
```

A total of **6,711** opportunities have reached a final outcome, Closed opportunities (Won+Lost, excluding Engaging/Prospecting). That high volume indicates a mature pipeline capable of generating meaningful trend analysis.

1.0.10 Basic Question 8

Which industry sector accounts for the highest number of client companies?

[9]:
```
# Companies by sector
sector_counts = accounts["sector"].value_counts()
display(sector_counts.to_frame(name="count"))

# Sector with the most companies
top_sector = sector_counts.idxmax()
print("Sector with most companies:", top_sector)
```

```
                    count
sector
retail                 17
technolgy              12
medical                12
marketing               8
finance                 8
```

```
software                7
entertainment           6
telecommunications      6
services                5
employment              4
```

```
Sector with most companies: retail
```

Retail accounts for the largest share of client companies, with technology and medical tied for second place. This shows a strong footprint in consumer-facing businesses but also diversification across industries.

1.0.11 Basic Question 9

Which products appear most frequently in closed deals?

[10]:
```
# Closed-deal product frequencies (Won + Lost)
product_deals = (sales_pipeline[sales_pipeline["deal_stage"].isin(["Won",␣
↪"Lost"])]
                 ["product"].value_counts().reset_index())
product_deals.columns = ["Product", "Number_of_Deals"]
display(product_deals)
```

```
          Product  Number_of_Deals
0       GTX Basic             1436
1      MG Special             1223
2          GTXPro             1147
3     MG Advanced             1084
4  GTX Plus Basic             1051
5    GTX Plus Pro              745
6         GTK 500               25
```

Among closed deals, **GTX Basic** and **MG Special** dominate, while **GTK 500** is rare. These two products are the main revenue drivers, showing where customer demand — and future focus — should be.

1.0.12 Basic Question 10

What is the overall win rate among closed opportunities?

[11]:
```
# Win/loss mix among closed opportunities (Won + Lost)
closed = sales_pipeline[sales_pipeline["deal_stage"].isin(["Won", "Lost"])]

win_loss_counts  = closed["deal_stage"].value_counts()
win_loss_percent = win_loss_counts / win_loss_counts.sum() * 100

display(win_loss_counts.to_frame(name="count"))
display(win_loss_percent.to_frame(name="percent"))
```

```
            count
deal_stage
```

```
Won          4238
Lost         2473

              percent
deal_stage
Won         63.150052
Lost        36.849948
```

About **63%** of closed deals were won, showing strong sales performance and effective deal qualification. The remaining **37%** lost share indicates potential to refine targeting and improve conversion further.

1.0.13 Basic Question 11

What are the earliest and latest close dates observed in the sales records?

[12]:
```
# Range of observed close dates
earliest = sales_pipeline["close_date"].min()
latest   = sales_pipeline["close_date"].max()
print("Earliest close date:", earliest)
print("Latest close date:", latest)
```

```
Earliest close date: 2017-03-01 00:00:00
Latest close date: 2017-12-31 00:00:00
```

The dataset spans from **March 2017** to **December 2017**. Any seasonal patterns identified should therefore be interpreted within this 10-month window.

1.0.14 Basic Question 12

How many sales representatives are assigned to each regional office?

[13]:
```
# Sales reps by regional office
region_counts = sales_teams["regional_office"].value_counts()
display(region_counts.to_frame(name="count"))
```

```
                 count
regional_office
East                12
West                12
Central             11
```

The sales team is evenly split across regions (12 reps each in East and West, 11 in Central), indicating no major coverage gaps and balanced resource allocation.

1.0.15 Basic Question 13

What is the average size of customer companies in terms of employee count?

[14]:
```
# Average employee count across customer companies
avg_employees = accounts["employees_numeric"].mean()
print("Average employees:", avg_employees)
```

```
Average employees: 4660.823529411765
```

Clients employ around **4,661** people on average, meaning most customers are mid-sized to large enterprises. This supports expectations of larger deal sizes and longer sales cycles.

1.0.16 Basic Question 14

How many customer companies are subsidiaries of other corporations?

[15]:
```
# Count how many accounts are subsidiaries of another company
num_subsidiaries = accounts["subsidiary_of"].notna().sum()
print("Number of subsidiaries:", num_subsidiaries)
```

```
Number of subsidiaries: 15
```

Fifteen companies are subsidiaries of larger corporations. These relationships may require negotiations at the parent-company level and provide cross-selling opportunities within corporate groups.

1.0.17 Intermediate Question 1

Which sales representative has achieved the highest number of successful deals, and what is the total value of those deals?

[16]:
```
# Top-performing sales agents by number of won deals and total closed value
won = sales_pipeline[sales_pipeline["deal_stage"] == "Won"]

aggr = (won.groupby("sales_agent")
           .agg(num_deals=("opportunity_id", "count"),
                total_value=("close_value_numeric", "sum")))

aggr_sorted = aggr.sort_values(by="total_value", ascending=False).reset_index()
display(aggr_sorted.head(5))
```

```
       sales_agent  num_deals  total_value
0  Darcel Schlecht        349    1153214.0
1   Vicki Laflamme        221     478396.0
2  Kary Hendrixson        209     454298.0
3     Cassey Cress        163     450489.0
4    Donn Cantrell        158     445860.0
```

Darcel Schlecht is the top performer, winning **349** deals worth about **$1.15 M**. Recognising and learning from this rep's practices could improve overall team performance.

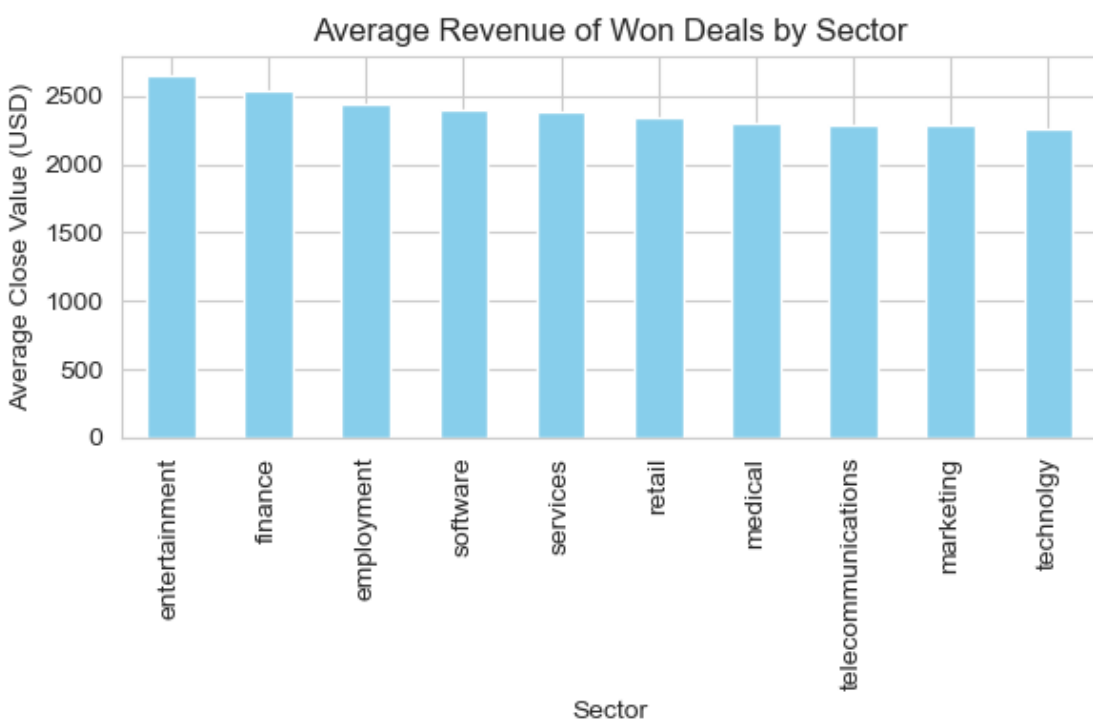
1.0.18 Intermediate Question 2

How does the average revenue from successful deals vary across different customer industry sectors?

[17]:
```
# Join won deals with account sector
won = sales_pipeline[sales_pipeline['deal_stage']=='Won'].
 ↪merge(accounts[['account','sector']], on='account', how='left')
```

```
sector_avg = won.groupby('sector')['close_value_numeric'].mean().
 ↪sort_values(ascending=False)
display(sector_avg.to_frame(name='avg_value'))
# Plot
plt.figure(figsize=(6,4))
sector_avg.plot(kind='bar', color='skyblue')
plt.title('Average Revenue of Won Deals by Sector')
plt.ylabel('Average Close Value (USD)')
plt.xlabel('Sector')
plt.tight_layout()
plt.show()
```

```
                     avg_value
sector
entertainment       2650.026923
finance             2535.754667
employment          2436.726257
software            2395.408889
services            2390.161435
retail              2337.331665
medical             2296.613176
telecommunications  2293.242105
marketing           2282.972772
technolgy           2258.549925
```



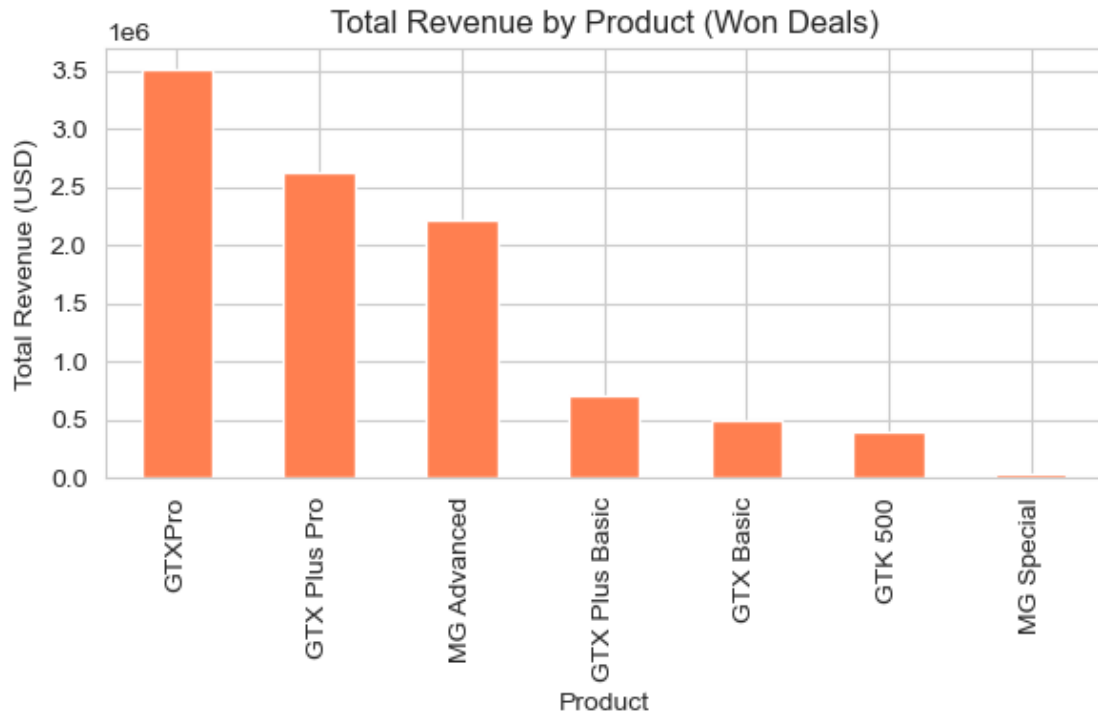
Average deal values are highest in entertainment/finance and lower in technology/marketing. Targeting the top sectors can raise revenue per deal without increasing volume.

1.0.19 Intermediate Question 3

Which product has generated the greatest total revenue from won opportunities?

[18]:
```
# Revenue by product from WON deals + bar chart
won = sales_pipeline[sales_pipeline["deal_stage"] == "Won"]

product_revenue = (won.groupby("product")["close_value_numeric"]
                      .sum()
                      .sort_values(ascending=False))

display(product_revenue.to_frame(name="total_revenue"))

plt.figure(figsize=(6,4))
product_revenue.plot(kind="bar", color="coral")
plt.title("Total Revenue by Product (Won Deals)")
plt.ylabel("Total Revenue (USD)")
plt.xlabel("Product")
plt.tight_layout()
plt.show()
```

```
                total_revenue
product
GTXPro              3510578.0
GTX Plus Pro        2629651.0
MG Advanced         2216387.0
GTX Plus Basic       705275.0
GTX Basic            499263.0
GTK 500              400612.0
MG Special            43768.0
```



GTXPro generates the most revenue (**$3.51 M**), followed by **GTX Plus Pro** and **MG Advanced**. Concentrating marketing and product development on these best-sellers could further amplify revenue.

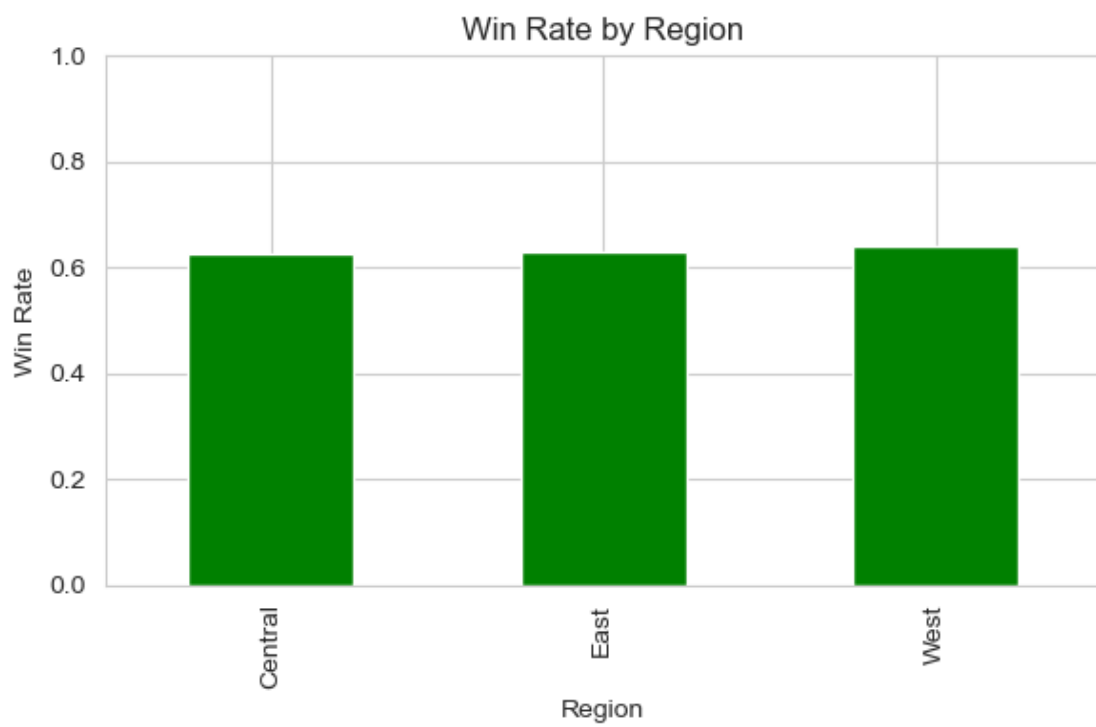
1.0.20 Intermediate Question 4

What is the win-rate of sales opportunities across each regional office?

[19]:
```
# Win/loss summary and win rate by regional office
closed = (sales_pipeline[sales_pipeline["deal_stage"].isin(["Won", "Lost"])]
          .merge(sales_teams[["sales_agent", "regional_office"]],␣
 ↪on="sales_agent", how="left"))

# Count Won/Lost deals per region and compute win rate
region_counts = (closed.groupby(["regional_office", "deal_stage"])
                       .size()
                       .unstack()
                       .fillna(0))
region_counts["WinRate"] = region_counts["Won"] / (region_counts["Won"] +␣
 ↪region_counts["Lost"])
display(region_counts[["Won", "Lost", "WinRate"]])

# Plot win rate by region
plt.figure(figsize=(6,4))
```

```
region_counts["WinRate"].plot(kind="bar", color="green")
plt.title("Win Rate by Region")
plt.ylabel("Win Rate")
plt.xlabel("Region")
plt.ylim(0,1)
plt.tight_layout()
plt.show()
```

```
deal_stage        Won  Lost   WinRate
regional_office
Central          1629   975  0.625576
East             1171   687  0.630248
West             1438   811  0.639395
```



Win rates are similar across regions, with the West slightly ahead. Sharing best practices from the West may help improve performance in other territories.

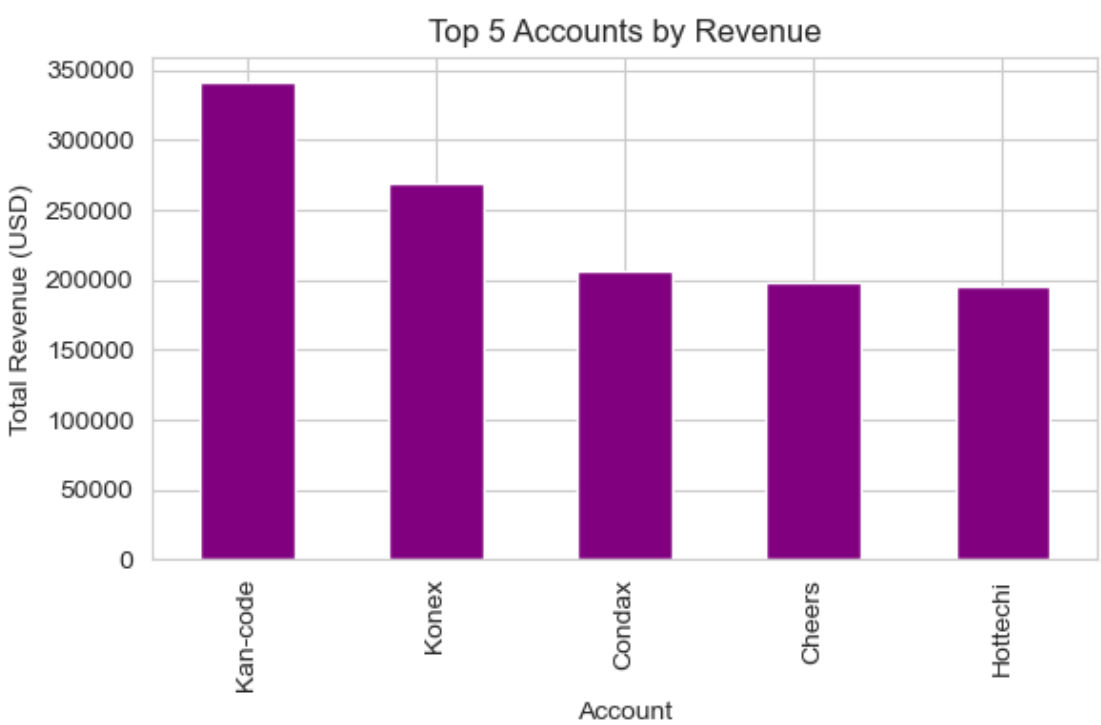
1.0.21 Intermediate Question 5

Which five customer companies have generated the highest cumulative revenue from closed deals?

[20]:
```
# Top 5 customer accounts by total revenue from WON deals
won = sales_pipeline[sales_pipeline["deal_stage"] == "Won"]
```

```
account_revenue = (won.groupby("account")["close_value_numeric"]
                      .sum()
                      .sort_values(ascending=False)
                      .head(5))

display(account_revenue.to_frame(name="total_revenue"))

# Plot top accounts by revenue
plt.figure(figsize=(6,4))
account_revenue.plot(kind="bar", color="purple")
plt.title("Top 5 Accounts by Revenue")
plt.ylabel("Total Revenue (USD)")
plt.xlabel("Account")
plt.tight_layout()
plt.show()
```

```
          total_revenue
account
Kan-code       341455.0
Konex          269245.0
Condax         206410.0
Cheers         198020.0
Hottechi       194957.0
```



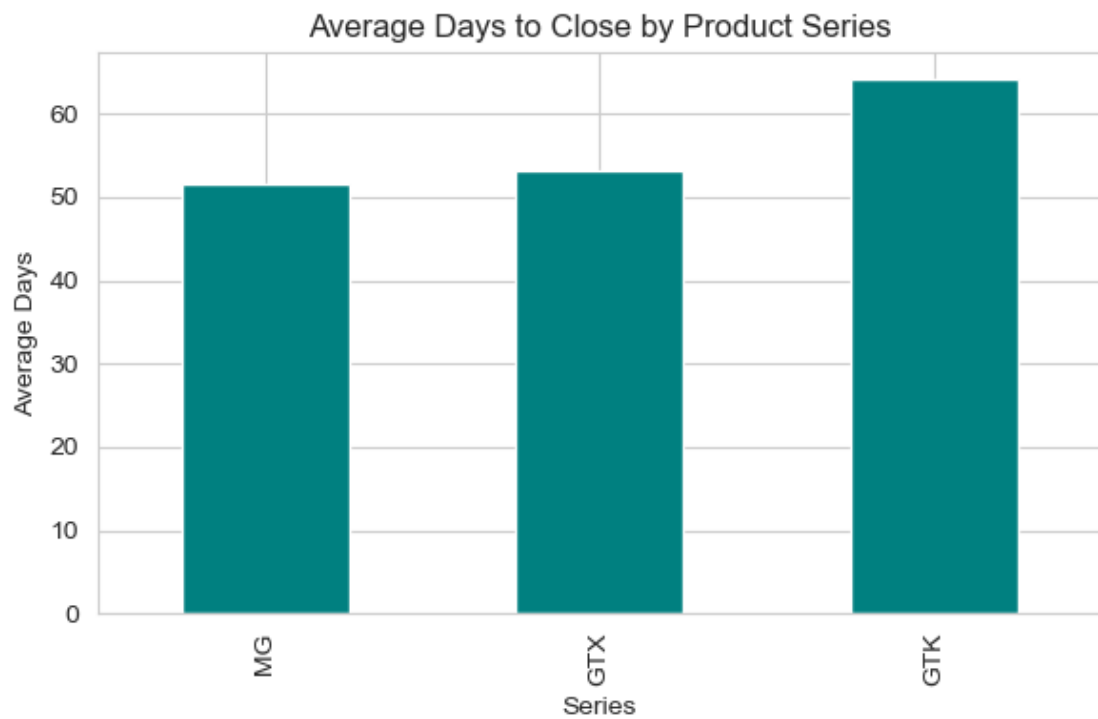
Top customers—**Kan-code**, **Konex**, **Condax**, **Cheers** and **Hottechi**—contribute significant revenue. Retaining and upselling these accounts should be a priority.

1.0.22 Intermediate Question 6

For each product family, what is the average time it takes to convert an engaged opportunity into a closed sale?

[21]:
```
# Average days to close by product series (Won deals only)
won = sales_pipeline[
    (sales_pipeline["deal_stage"] == "Won")
    & (sales_pipeline["close_date"].notna())
    & (sales_pipeline["engage_date"].notna())
]

# Add product series info
won = won.merge(products[["product", "series"]], on="product", how="left")

# Calculate duration between engagement and close
won["days_to_close"] = (won["close_date"] - won["engage_date"]).dt.days

# Average closing time per product series
series_avg_days = (won.groupby("series")["days_to_close"]
                      .mean()
                      .sort_values())

display(series_avg_days.to_frame(name="avg_days"))

# Plot average close time by series
plt.figure(figsize=(6,4))
series_avg_days.plot(kind="bar", color="teal")
plt.title("Average Days to Close by Product Series")
plt.ylabel("Average Days")
plt.xlabel("Series")
plt.tight_layout()
plt.show()
```

```
         avg_days
series
MG      51.539046
GTX     53.094284
GTK     64.066667
```



MG series deals close quickest (around **52 days**), **GTX** follows (**53 days**) and **GTK** takes longest (**64 days**). Understanding why GTK cycles are slower (complexity, fewer reps) could help shorten them.

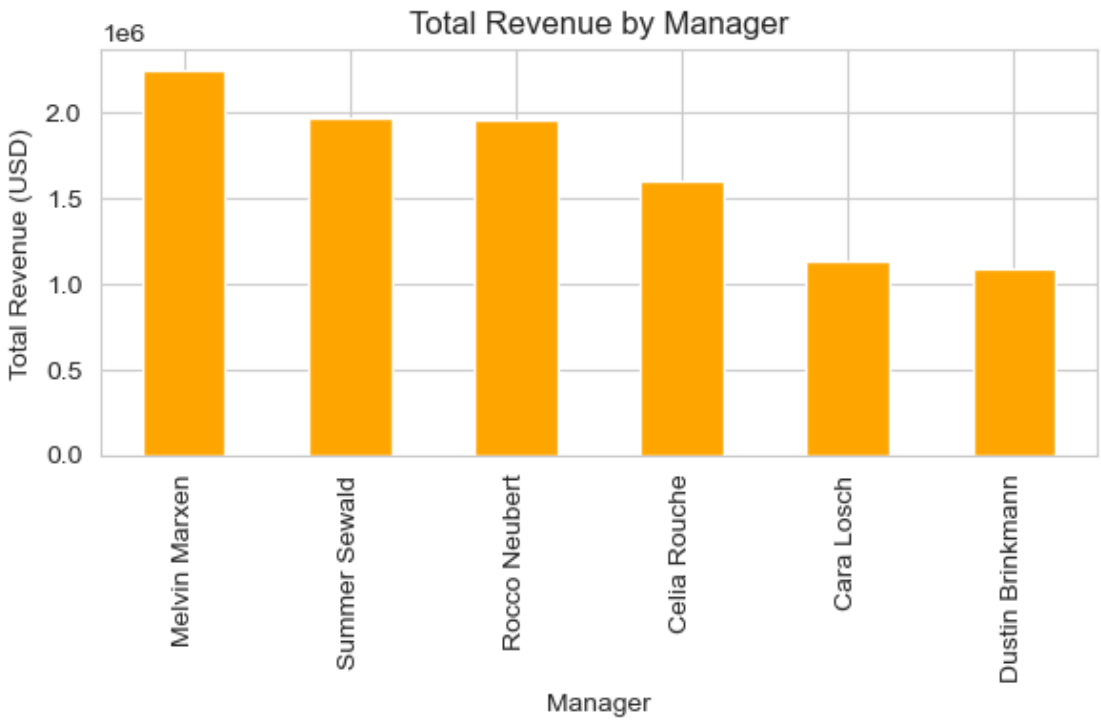
1.0.23 Intermediate Question 7

Which sales manager's team has produced the highest total revenue from successful deals?

[22]:
```
# Total revenue by sales manager (Won deals only)
won = (sales_pipeline[sales_pipeline["deal_stage"] == "Won"]
       .merge(sales_teams[["sales_agent", "manager"]], on="sales_agent",␣
 ↪how="left"))

manager_revenue = (won.groupby("manager")["close_value_numeric"]
                      .sum()
                      .sort_values(ascending=False))

display(manager_revenue.to_frame(name="total_revenue"))

# Plot total revenue by manager
plt.figure(figsize=(6,4))
manager_revenue.plot(kind="bar", color="orange")
plt.title("Total Revenue by Manager")
```

```
plt.ylabel("Total Revenue (USD)")
plt.xlabel("Manager")
plt.tight_layout()
plt.show()
```

```
                  total_revenue
manager
Melvin Marxen         2251930.0
Summer Sewald         1964750.0
Rocco Neubert         1960545.0
Celia Rouche          1603897.0
Cara Losch            1130049.0
Dustin Brinkmann      1094363.0
```



Melvin Marxen's team produces the most revenue (~**$2.25 M**), followed by **Summer Sewald** and **Rocco Neubert**. Their management practices might serve as a model for other teams.

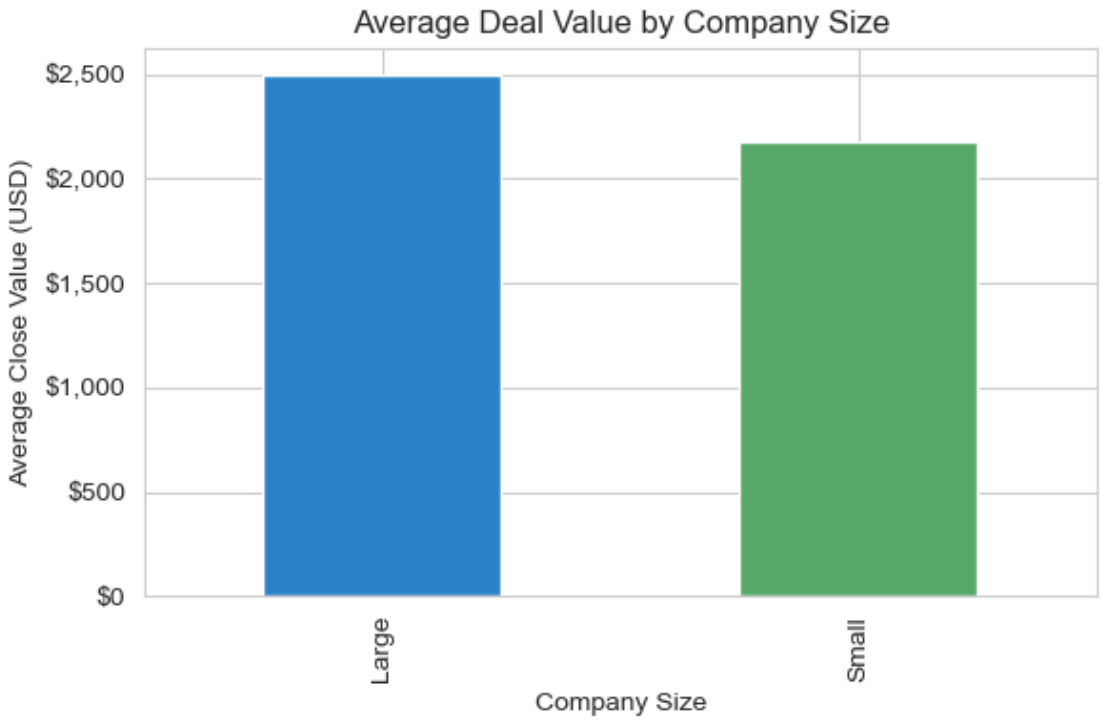
1.0.24 Intermediate Question 8

Do companies with smaller headcounts or larger headcounts generate higher average deal values?

[43]:
```
# Categorize accounts by size using median employee count
median_emp = accounts["employees_numeric"].median()
```

```
accounts["size_category"] = np.where(accounts["employees_numeric"] < median_emp,␣
↪"Small", "Large")

# Join won deals with company size
won = (sales_pipeline[sales_pipeline["deal_stage"] == "Won"]
       .merge(accounts[["account", "size_category"]], on="account", how="left"))

# Compute average closed deal value by company size
avg_by_size = (won.groupby("size_category")["close_value_numeric"]
                  .mean()
                  .sort_index())

# Display formatted table
avg_by_size_tbl = avg_by_size.to_frame(name="Average Close Value (USD)")
display(avg_by_size_tbl)
# Plot average deal value by company size
import matplotlib.ticker as mtick
plt.figure(figsize=(6,4))
ax = avg_by_size.plot(kind="bar", color=["#2C82C9", "#55A868"])
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter("${x:,.0f}"))
ax.set_title("Average Deal Value by Company Size")
ax.set_xlabel("Company Size")
ax.set_ylabel("Average Close Value (USD)")
plt.tight_layout()
plt.show()
```

```
               Average Close Value (USD)
size_category
Large                        2497.956414
Small                        2176.358804
```



Large companies average **~$2.50k** per won deal versus **~$2.18k** for small ones — a difference of about **$320**.
Targeting larger clients modestly increases deal value and may enhance overall revenue growth.

1.0.25 Intermediate Question 9

How have the number of successful deals changed month by month over the period covered by the data?

[24]:
```
# Won deals trend over time (monthly)
won = sales_pipeline.loc[sales_pipeline["deal_stage"] == "Won"].copy()

# Extract month from close date
won["month"] = won["close_date"].dt.to_period("M")

# Count won deals per month
monthly_counts = won.groupby("month").size()
display(monthly_counts.to_frame(name="count"))

# Plot monthly won deal counts
plt.figure(figsize=(8,4))
monthly_counts.plot(kind="line", marker="o", color="navy")
plt.title("Won Deals per Month")
plt.ylabel("Number of Won Deals")
```

```
plt.xlabel("Month")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
         count
month
2017-03    531
2017-04    285
2017-05    438
2017-06    531
2017-07    308
2017-08    446
2017-09    503
2017-10    279
2017-11    406
2017-12    511
```

Won deals peak in **March** and **June** (531 each) and remain strong in **September** and **December**. April and October are quieter months. Align marketing and staffing efforts with these seasonal trends.

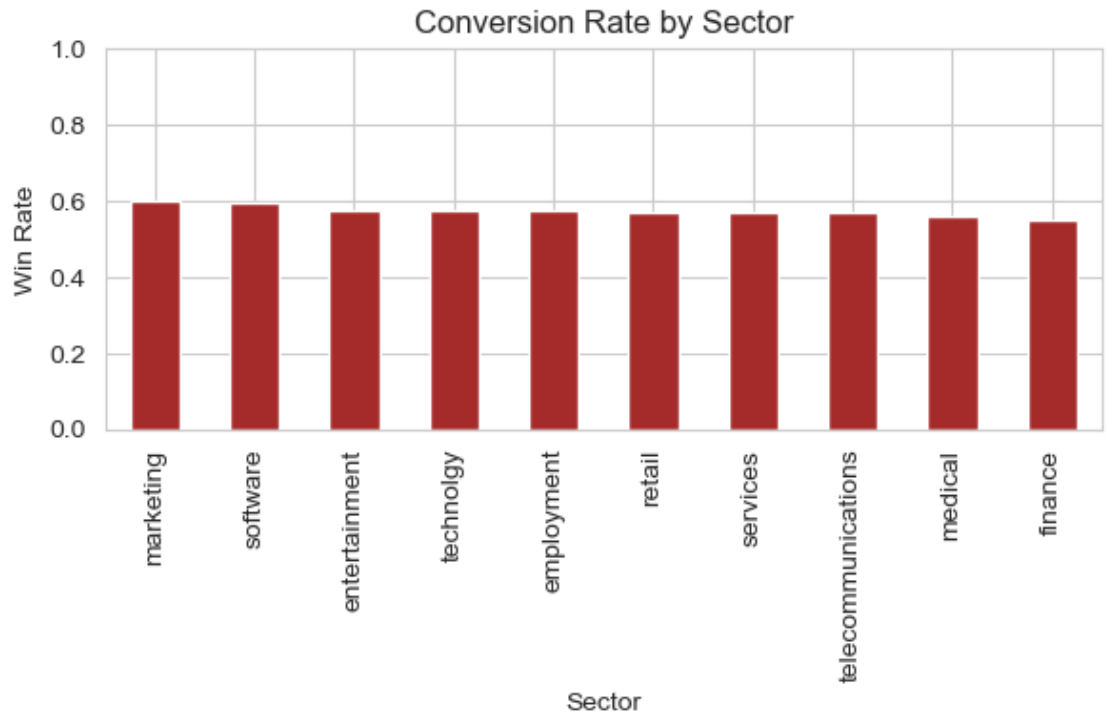
1.0.26 Intermediate Question 10

Which industry sectors have the highest success rate in converting prospects into closed sales?

[25]:
```
# Conversion (win) rate by customer sector
merged = sales_pipeline.merge(accounts[["account", "sector"]], on="account",␣
 ↪how="left")
```

```python
# Total and won deals per sector
total_by_sector = merged.groupby("sector").size()
won_by_sector   = merged[merged["deal_stage"] == "Won"].groupby("sector").size()

# Compute and display conversion rate
conversion_rate = (won_by_sector / total_by_sector).sort_values(ascending=False)
display(conversion_rate.to_frame(name="win_rate"))

# Plot conversion rate by sector
plt.figure(figsize=(6,4))
conversion_rate.plot(kind="bar", color="brown")
plt.title("Conversion Rate by Sector")
plt.ylabel("Win Rate")
plt.xlabel("Sector")
plt.ylim(0,1)
plt.tight_layout()
plt.show()
```

```
                    win_rate
sector
marketing           0.599407
software            0.594452
entertainment       0.576497
technolgy           0.575966
employment          0.575563
retail              0.571940
services            0.571795
telecommunications  0.568862
medical             0.563273
finance             0.553097
```



Marketing and software sectors show the highest conversion rates from prospect to win (around **60 %**). Industries like medical and finance convert slightly less efficiently. Sales strategies should prioritise high-converting sectors.

1.0.27 Intermediate Question 11

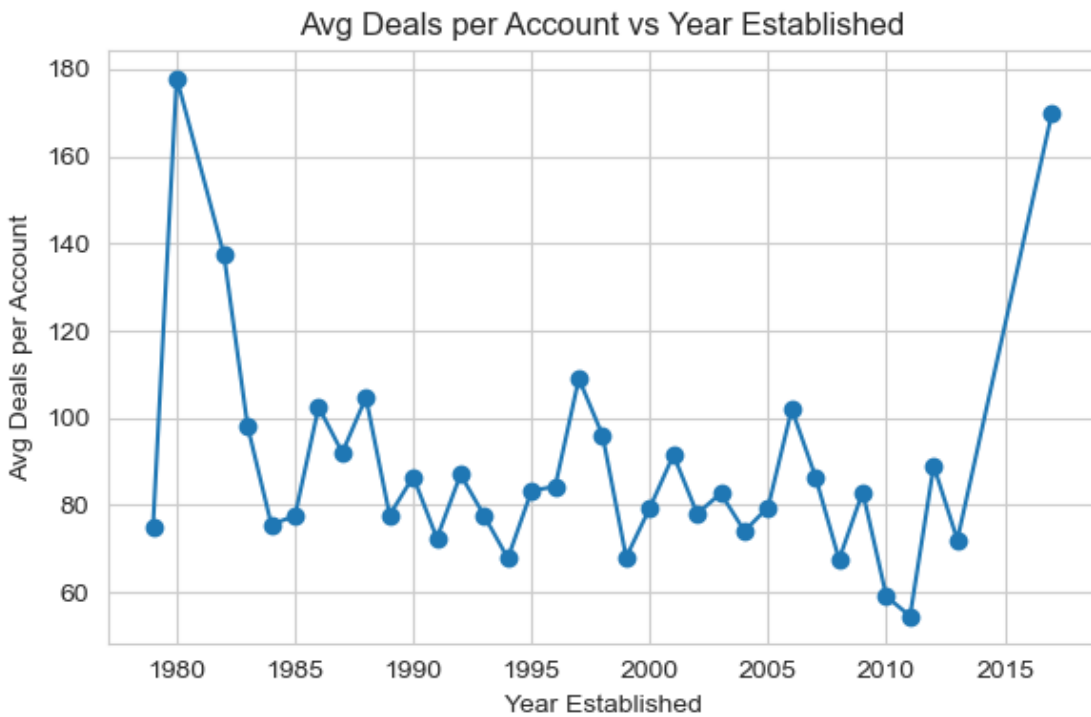
Is there a relationship between the year a customer company was founded and the number of deals it has been involved in?

[26]:
```
# Average number of deals per account by founding year
deals_per_acct = (sales_pipeline.groupby("account")
                  .size()
                  .rename("deal_count")
                  .reset_index())

# Merge with account founding year
acct_year = (accounts[["account", "year_established"]]
             .merge(deals_per_acct, on="account", how="left")
             .dropna(subset=["year_established"]))

# Compute average deals per account per founding year
year_avg = (acct_year.groupby("year_established")["deal_count"]
            .mean()
            .sort_index())
```

```python
display(year_avg.to_frame(name="avg_deals_per_account"))

# Quantify correlation between company age and deal activity
from scipy.stats import pearsonr
yrs = year_avg.index.values.astype(float)
avg = year_avg.values.astype(float)
r, p = pearsonr(yrs, avg)
print(f"Pearson r = {r:.2f} (p={p:.3g})")

# Plot trend between founding year and deal activity
plt.figure(figsize=(6,4))
plt.plot(yrs, avg, marker="o")
plt.title("Avg Deals per Account vs Year Established")
plt.xlabel("Year Established")
plt.ylabel("Avg Deals per Account")
plt.tight_layout()
plt.show()
```

```
                  avg_deals_per_account
year_established
1979                          75.000000
1980                         178.000000
1982                         137.333333
1983                          98.000000
1984                          75.500000
1985                          77.500000
1986                         102.500000
1987                          92.000000
1988                         104.666667
1989                          77.666667
1990                          86.500000
1991                          72.333333
1992                          87.000000
1993                          77.500000
1994                          67.750000
1995                          83.250000
1996                          84.250000
1997                         109.000000
1998                          96.000000
1999                          68.000000
2000                          79.333333
2001                          91.500000
2002                          78.000000
2003                          82.666667
2004                          74.000000
2005                          79.500000
2006                         102.000000
```

```
2007                          86.500000
2008                          67.500000
2009                          83.000000
2010                          59.000000
2011                          54.500000
2012                          89.000000
2013                          72.000000
2017                         170.000000

Pearson r = -0.20 (p=0.257)
```



By averaging **deals per account** within each founding year, we remove the bias from having more or fewer clients in a given year.
The trend line and correlation (printed above) indicate the relationship between company age and deal activity is **weak**; founding year does **not** strongly predict how many deals a client does.
This suggests sales potential is driven more by **account-specific factors** (fit, product needs, engagement) than by how old the company is.

1.0.28 Intermediate Question 12

How do average deal values and win rates compare between companies founded before the year 2000 and those founded from 2000 onward?

[27]:

```
# Compare deal performance for companies founded before vs after 2000
accounts["era"] = np.where(accounts["year_established"] < 2000, "Before 2000",␣
↪"2000 or later")

# Merge sales data with company era
merged = sales_pipeline.merge(accounts[["account", "era"]], on="account",␣
↪how="left")

# Compute average closed deal value for won deals
won = merged[merged["deal_stage"] == "Won"]
won_avg = won.groupby("era")["close_value_numeric"].mean()

# Compute win rate by era
total_counts = merged.groupby("era").size()
won_counts   = merged[merged["deal_stage"] == "Won"].groupby("era").size()
win_rates    = won_counts / total_counts

# Display results
display(won_avg.to_frame(name="avg_close_value"))
display(win_rates.to_frame(name="win_rate"))
```

```
               avg_close_value
era
2000 or later      2312.999244
Before 2000        2382.653859

               win_rate
era
2000 or later  0.562979
Before 2000    0.580100
```

Accounts founded before 2000 have slightly higher average deal values and win rates than those established later. Long-established companies may therefore be easier to close and yield higher value.

1.0.29 Intermediate Question 13

Which product family performs best in each region when measured by total revenue from closed deals?

[45]:
```
# Identify top-performing product series by region (Won deals only)
won = sales_pipeline[sales_pipeline["deal_stage"] == "Won"]

# Merge with region and product series info
merged = (won.merge(sales_teams[["sales_agent", "regional_office"]],␣
↪on="sales_agent", how="left")
             .merge(products[["product", "series"]], on="product", how="left"))

# Total revenue by region and series
```

```
region_series = (merged.groupby(["regional_office",␣
↪"series"])["close_value_numeric"]
                        .sum()
                        .reset_index())

# Select top series per region
top_series = (region_series.sort_values(["regional_office",␣
↪"close_value_numeric"],
                                        ascending=[True, False])
                              .drop_duplicates("regional_office"))

# Display formatted result
# Make a Series keyed by office & series, then to_frame -> PDF-safe
t = top_series.rename(columns={"close_value_numeric": "Total Revenue (USD)"}).
↪copy()
t["Total Revenue (USD)"] = (
    t["Total Revenue (USD)"].round(0).map("${:,.0f}".format)
)
display(t)
```

```
  regional_office series Total Revenue (USD)
0         Central    GTX          $1,391,255
2            East    GTX          $1,140,178
5            West    GTX          $1,302,756
```

In all regions, the **GTX** family is the top-performing product line, underscoring its universal appeal. Focusing on this series across regions could maximise sales.

1.0.30 Intermediate Question 14

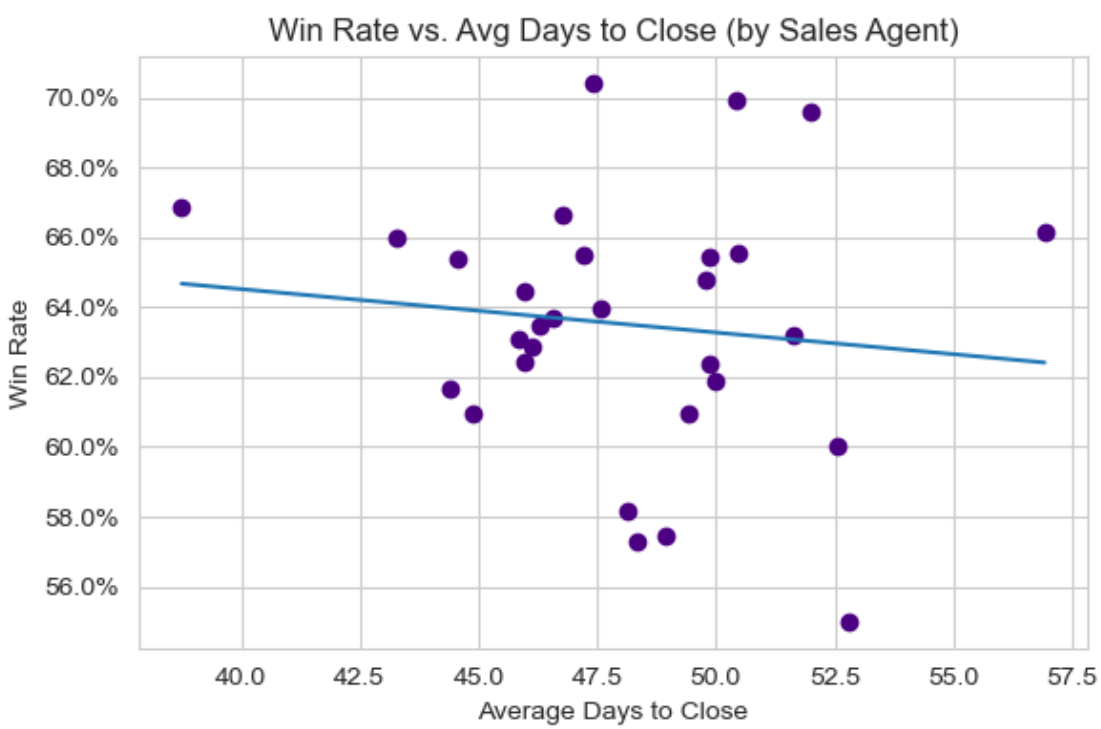
Among sales representatives, what is the average number of days to close a deal, and how does this correlate with their win rate?

[48]:
```
# Relationship between deal speed and win rate (per sales agent)
closed = sales_pipeline[sales_pipeline["deal_stage"].isin(["Won", "Lost"])].
↪copy()

# Ensure valid dates and compute days to close
closed["engage_date"] = pd.to_datetime(closed["engage_date"], errors="coerce")
closed["close_date"]  = pd.to_datetime(closed["close_date"],  errors="coerce")
closed = closed.dropna(subset=["engage_date", "close_date"])
closed["days_to_close"] = (closed["close_date"] - closed["engage_date"]).dt.days

# Calculate average duration and win rate per agent
avg_duration = closed.groupby("sales_agent")["days_to_close"].mean()
agent_total  = closed.groupby("sales_agent")["deal_stage"].size()
agent_won    = closed["deal_stage"].eq("Won").groupby(closed["sales_agent"]).
↪sum()
```

```
win_rate     = agent_won / agent_total

# Combine metrics into one dataframe
metrics = pd.concat([avg_duration, win_rate], axis=1).reset_index()
metrics.columns = ["sales_agent", "avg_days_to_close", "win_rate"]

# Display metrics sorted by win rate
df_sorted = (
    metrics.sort_values("win_rate", ascending=False)
           .rename(columns={"avg_days_to_close": "Avg Days to Close",
                            "win_rate": "Win Rate"})
           .reset_index(drop=True)
)

df_display = df_sorted.copy()
df_display["Avg Days to Close"] = df_display["Avg Days to Close"].map(lambda x:␣
↪f"{x:,.1f}")
df_display["Win Rate"] = df_display["Win Rate"].map(lambda x: f"{x:.1%}")
display(df_display)   # no .style


# Correlation analysis and visualization
from scipy.stats import pearsonr
r, p = pearsonr(metrics["avg_days_to_close"], metrics["win_rate"])
print(f"Pearson r = {r:.2f} (p={p:.3g})")

plt.figure(figsize=(6,4))
plt.scatter(metrics["avg_days_to_close"], metrics["win_rate"], color="indigo")
m, b = np.polyfit(metrics["avg_days_to_close"], metrics["win_rate"], 1)
x = np.linspace(metrics["avg_days_to_close"].min(), metrics["avg_days_to_close"].
↪max(), 100)
plt.plot(x, m*x + b)
plt.gca().yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
plt.title("Win Rate vs. Avg Days to Close (by Sales Agent)")
plt.xlabel("Average Days to Close")
plt.ylabel("Win Rate")
plt.tight_layout()
plt.show()
```

```
           sales_agent Avg Days to Close Win Rate
0        Hayden Neloms              47.4    70.4%
1      Maureen Marcano              50.4    70.0%
2       Wilburn Farren              52.0    69.6%
3       Cecily Lampkin              38.7    66.9%
4    Versie Hillebrand              46.8    66.7%
5          Moses Frase              56.9    66.2%
6            Boris Faz              43.3    66.0%
```

```
7        James Ascencio              50.5    65.5%
8         Corliss Cosme              47.2    65.5%
9       Rosalina Dieter              49.8    65.5%
10         Reed Clapper              44.6    65.4%
11   Jonathan Berthelot              49.8    64.8%
12  Rosie Papadopoulos               46.0    64.5%
13       Kami Bicknell              47.6    64.0%
14      Vicki Laflamme              46.6    63.7%
15         Elease Gluck              46.3    63.5%
16    Violet Mclelland               51.7    63.2%
17     Darcel Schlecht               45.8    63.1%
18   Marty Freudenburg               46.1    62.9%
19         Cassey Cress              46.0    62.5%
20     Kary Hendrixson               49.9    62.4%
21       Anna Snelling               50.0    61.9%
22           Zane Levy               44.4    61.7%
23       Garret Kinder               49.4    61.0%
24     Daniell Hammack               44.9    61.0%
25     Niesha Huffines               52.6    60.0%
26    Gladys Colclough               48.1    58.2%
27       Donn Cantrell               48.9    57.5%
28      Markita Hansen               48.3    57.3%
29     Lajuana Vencill               52.8    55.0%

Pearson r = -0.12 (p=0.536)
```



The correlation between average days to close and win rate is **r = -0.12 (p = 0.54)**, showing *no significant relationship*.
This means faster closings do **not necessarily** lead to higher win rates in this dataset.
Performance improvements may depend more on deal quality and customer fit than on speed alone.

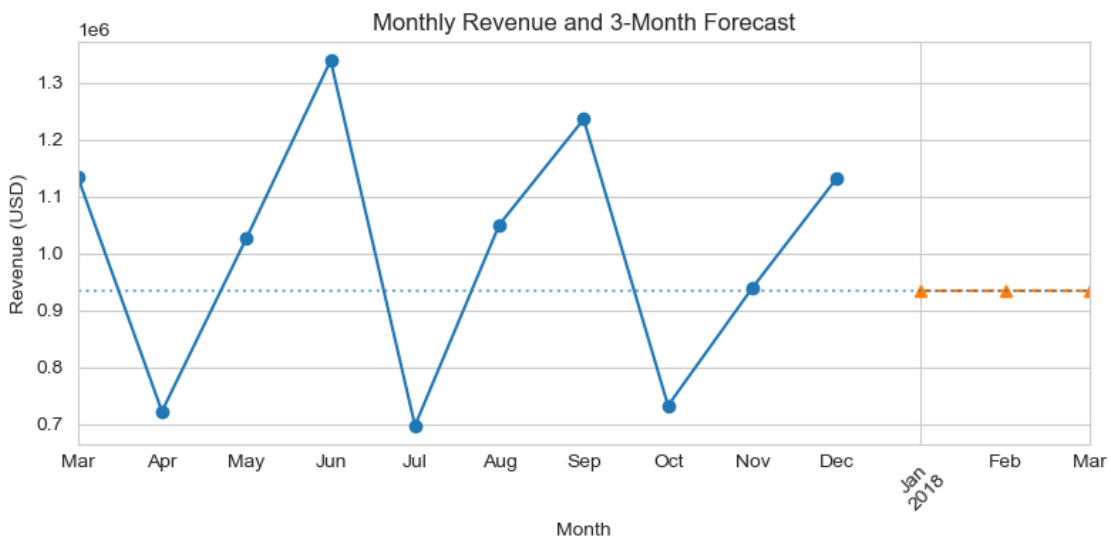
1.0.31 Advanced Question 1

Based on historical monthly revenues from closed deals, what do forecasts predict for the next three months?

[50]:
```
# Build monthly revenue (WON only)
won = sales_pipeline[sales_pipeline["deal_stage"] == "Won"].copy()
won["month"] = won["close_date"].dt.to_period("M")
monthly_rev = (won.groupby("month")["close_value_numeric"]
                  .sum()
                  .sort_index())

# Naïve forecast: repeat the average of the last 3 months for the next 3 months
last_three_avg  = monthly_rev.tail(3).mean()
forecast_index  = pd.period_range(start=monthly_rev.index[-1] + 1, periods=3,␣
 ↪freq="M")
forecast_values = pd.Series(last_three_avg, index=forecast_index)

# Display formatted tables
display(monthly_rev.rename("Revenue (USD)").round(0).map("${:,.0f}".format).
 ↪to_frame())
display(forecast_values.rename("Forecast (USD)").round(0).map("${:,.0f}".format).
 ↪to_frame())

# Plot historical + forecast
hist = monthly_rev.copy();            hist.index  = hist.index.to_timestamp()
fcast = forecast_values.copy();       fcast.index = fcast.index.to_timestamp()

ax = hist.plot(kind="line", marker="o", label="Historical", figsize=(8,4))
fcast.plot(ax=ax, marker="^", linestyle="--", label="Forecast")
ax.axhline(fcast.iloc[0], linestyle=":", alpha=0.6)  # visual guide for forecast␣
 ↪level
ax.set_title("Monthly Revenue and 3-Month Forecast")
ax.set_ylabel("Revenue (USD)")
ax.set_xlabel("Month")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
        Revenue (USD)
month
2017-03    $1,134,672
```

```
2017-04      $721,932
2017-05    $1,025,713
2017-06    $1,338,466
2017-07      $696,932
2017-08    $1,050,059
2017-09    $1,235,264
2017-10      $731,980
2017-11      $938,943
2017-12    $1,131,573

        Forecast (USD)
2018-01       $934,165
2018-02       $934,165
2018-03       $934,165
```



Monthly revenue is forecast at **~$0.934M per month for Jan–Mar 2018**, using a simple trailing average of the last three months (Oct–Dec 2017). Historical peaks occur in **June (~$1.35M)** and **September (~$1.24M)**, with softer months like **April** and **October**. Given only ~10 months of data and no seasonal adjustment, treat this as a **baseline (naïve) projection** for short-term planning.

1.0.32 Advanced Question 2

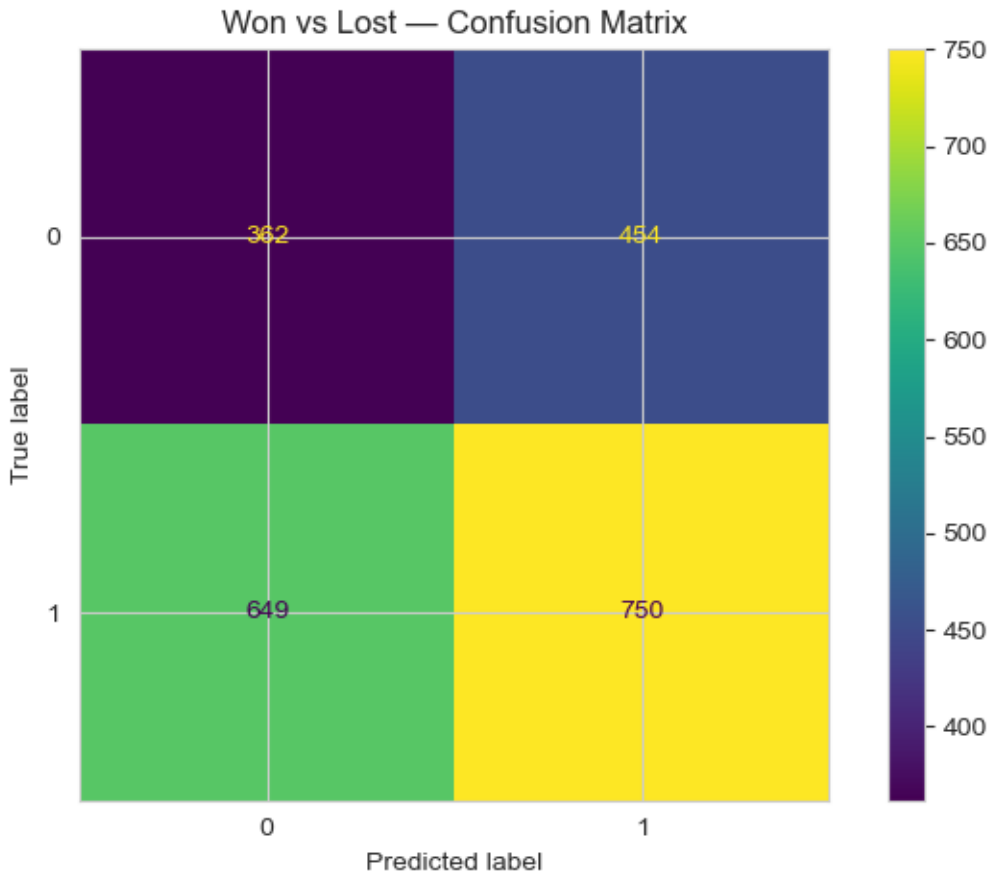
Which factors most influence whether a sales opportunity is won or lost?

[51]:
```
# Imports (minimal; safe if already imported)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
```

```python
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,␣
↪roc_auc_score, ConfusionMatrixDisplay

# Prepare data (Won/Lost only) and join features
df = sales_pipeline.loc[sales_pipeline['deal_stage'].isin(['Won','Lost'])].copy()
df = df.
↪merge(accounts[['account','sector','revenue_numeric','employees_numeric']],␣
↪on='account', how='left')
df = df.merge(products[['product','series']], on='product', how='left')
df = df.merge(sales_teams[['sales_agent','regional_office']], on='sales_agent',␣
↪how='left')

y = (df['deal_stage'] == 'Won').astype(int).values
X =␣
↪df[['sector','series','regional_office','revenue_numeric','employees_numeric']].
↪copy()

num_cols = ['revenue_numeric','employees_numeric']
cat_cols = ['sector','series','regional_office']

# Version-safe OneHotEncoder (handles sklearn >=1.4 and older)
try:
    ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    ohe = OneHotEncoder(handle_unknown='ignore', sparse=False)

prep = ColumnTransformer(
    transformers=[
        ('num', Pipeline([('imp', SimpleImputer(strategy='median')),
                          ('sc', StandardScaler())]), num_cols),
        ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                          ('oh', ohe)]), cat_cols),
    ],
    remainder='drop'
)

clf = Pipeline(steps=[
    ('prep', prep),
    ('logit', LogisticRegression(solver='lbfgs', max_iter=2000,␣
↪class_weight='balanced'))
])

# Train/test and fit
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42, stratify=y
```

```python
)
clf.fit(X_train, y_train)

# Evaluate
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1]

print('Accuracy :', f'{accuracy_score(y_test, y_pred):.3f}')
print('ROC-AUC  :', f'{roc_auc_score(y_test, y_prob):.3f}')
print(classification_report(y_test, y_pred, zero_division=0))

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title('Won vs Lost - Confusion Matrix')
plt.tight_layout(); plt.show()

# Show most influential features (coefficients)
feat_names = clf.named_steps['prep'].get_feature_names_out()
coefs = clf.named_steps['logit'].coef_[0]
coef_df = pd.DataFrame({'feature': feat_names, 'coef': coefs}).
 ↪sort_values('coef', ascending=False)
t = coef_df.copy()
t["coef"] = t["coef"].map("{:+.3f}".format)
display(t.head(10))
display(t.tail(10))
```

```
Accuracy : 0.502
ROC-AUC  : 0.487
              precision    recall  f1-score   support

           0       0.36      0.44      0.40       816
           1       0.62      0.54      0.58      1399

    accuracy                           0.50      2215
   macro avg       0.49      0.49      0.49      2215
weighted avg       0.53      0.50      0.51      2215
```




```
                     feature    coef
10      cat__sector_technolgy  +0.114
8        cat__sector_services  +0.101
5       cat__sector_marketing  +0.095
9        cat__sector_software  +0.064
17  cat__regional_office_West  +0.034
6         cat__sector_medical  +0.021
1      num__employees_numeric  +0.019
16  cat__regional_office_East  +0.014
12            cat__series_GTK  +0.002
14             cat__series_MG  -0.001
```

```
                           feature    coef
12                 cat__series_GTK  +0.002
14                  cat__series_MG  -0.001
7               cat__sector_retail  -0.001
13                 cat__series_GTX  -0.006
0             num__revenue_numeric  -0.038
3        cat__sector_entertainment  -0.041
15    cat__regional_office_Central  -0.053
```

```
11  cat__sector_telecommunications  -0.079
2           cat__sector_employment  -0.137
4              cat__sector_finance  -0.142
```

Performance:
Accuracy **0.502**, ROC-AUC **0.487** → **below a simple majority baseline (~0.63)** and close to random.
The model is **not reliable for prediction** with the current features.

Signals (interpret cautiously due to weak model):
Positive coefficients (higher odds of **Win**) appear for **Technology**, **Services**, and **Marketing** sectors, and the **West** region.
Negative effects (toward **Loss**) appear for **Finance**, **Employment**, and **Telecommunications** sectors, and the **Central** region.
Account size proxies (revenue, employees) have **minimal influence**.

Action:
Treat these patterns as **exploratory** rather than causal.
To improve model performance:
1. Add richer features (e.g., month/quarter, product line, manager, days_to_close, stage history).
2. Use **non-linear models** (e.g., Gradient Boosting, XGBoost) and tune probability thresholds.
3. Validate with **time-based splits** and focus on **ROC-AUC / PR-AUC** for better evaluation.

1.0.33 Advanced Question 3

Can customer companies be grouped into distinct segments using characteristics like revenue, employee size and industry? What distinguishes each segment?

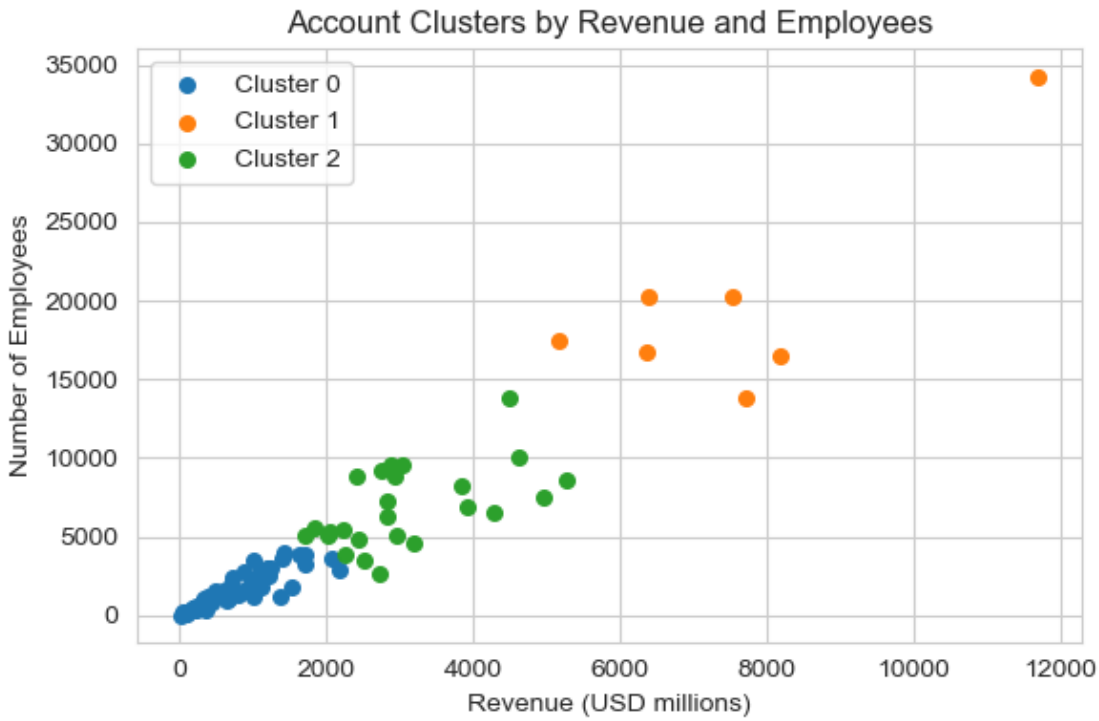
[52]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# 1) Select features (revenue, employees) and keep sector for labeling
feat = accounts[['revenue_numeric', 'employees_numeric', 'sector']].copy()

# Minimal cleaning: impute numerics (rare NaNs) with median
imp = SimpleImputer(strategy='median')
num = imp.fit_transform(feat[['revenue_numeric', 'employees_numeric']])

# 2) Scale numerics for K-Means
scaler = StandardScaler()
X = scaler.fit_transform(num)

# 3) Fit K-Means (stable init)
kmeans = KMeans(n_clusters=3, n_init=20, random_state=42)
clusters = kmeans.fit_predict(X)
```

```
feat = feat.assign(cluster=clusters)

# 4) Quality check (silhouette)
sil = silhouette_score(X, clusters)
print(f"Silhouette score: {sil:.2f}")

# 5) Summaries
cluster_summary = (
    feat.groupby('cluster',␣
 ↪as_index=False)[['revenue_numeric','employees_numeric']].mean()
    .rename(columns={'revenue_numeric':'avg_revenue','employees_numeric':
 ↪'avg_employees'})
)
dominant_sector = (
    feat.groupby('cluster')['sector'].agg(lambda s: s.value_counts().idxmax()).
 ↪reset_index(name='dominant_sector')
)

cs = cluster_summary.copy()
cs['avg_revenue']  = cs['avg_revenue'].round(0).map('{:,.0f}'.format)
cs['avg_employees'] = cs['avg_employees'].round(0).map('{:,.0f}'.format)
display(cs)
display(dominant_sector)

# 6) Plot clusters (revenue vs employees)
plt.figure(figsize=(6,4))
for c in sorted(feat['cluster'].unique()):
    sub = feat[feat['cluster']==c]
    plt.scatter(sub['revenue_numeric'], sub['employees_numeric'],␣
 ↪label=f'Cluster {c}', s=30)
plt.xlabel('Revenue (USD millions)')
plt.ylabel('Number of Employees')
plt.title('Account Clusters by Revenue and Employees')
plt.legend()
plt.tight_layout()
plt.show()
```

```
Silhouette score: 0.59
```

	cluster	avg_revenue	avg_employees
0	0	748	1,601
1	1	7,575	19,910
2	2	3,074	6,879

	cluster	dominant_sector
0	0	retail
1	1	technolgy
2	2	retail



The clustering reveals three distinct client segments.
Cluster 0 (SMB): ~$0.75B revenue, ~1.6k employees — mainly Retail.
Cluster 2 (Mid-market): ~$3.1B revenue, ~6.9k employees — also Retail.
Cluster 1 (Enterprise): ~$7.6B revenue, ~19.9k employees — dominated by Technology firms.

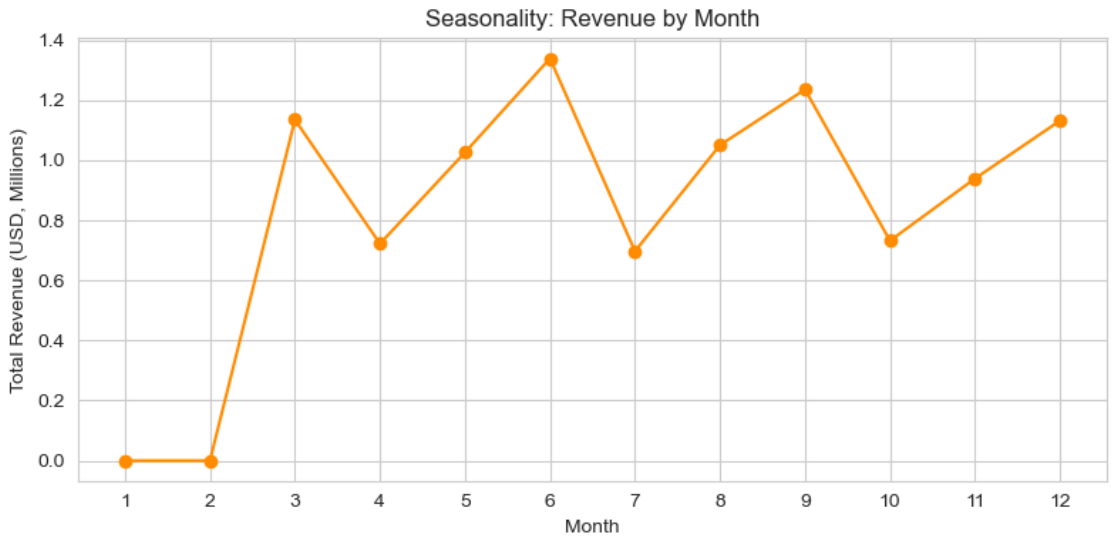
These patterns show a clear size-based structure: smaller retail accounts, midsized mixed clients, and large technology enterprises.
Sales and marketing strategies can be tailored by segment—value offerings for SMBs, cross-sell for midsize clients, and enterprise packages for large tech firms.

1.0.34 Advanced Question 4

Are there any seasonal patterns in the data? For example, do certain months consistently show higher or lower revenues or different proportions of deals in each pipeline stage?

[33]:
```
# Monthly revenue & won counts (WON only)
won = sales_pipeline[sales_pipeline['deal_stage'] == 'Won'].copy()
won['month'] = won['close_date'].dt.month

monthly_rev  = won.groupby('month')['close_value_numeric'].sum()
monthly_cnt  = won.groupby('month').size()

# Ensure axis shows all months 1..12 (fill missing Jan/Feb with 0)
idx = pd.Index(range(1, 13), name='month')
```

```
monthly_rev = monthly_rev.reindex(idx, fill_value=0)
monthly_cnt = monthly_cnt.reindex(idx, fill_value=0)

# Plot revenue (line)
plt.figure(figsize=(8,4))
(monthly_rev/1e6).plot(kind='line', marker='o', color='darkorange')
plt.title('Seasonality: Revenue by Month')
plt.ylabel('Total Revenue (USD, Millions)')
plt.xlabel('Month')
plt.xticks(range(1,13))
plt.tight_layout()
plt.show()

# Plot counts (bar)
plt.figure(figsize=(8,4))
monthly_cnt.plot(kind='bar', color='steelblue')
plt.title('Seasonality: Number of Won Deals by Month')
plt.ylabel('Number of Won Deals')
plt.xlabel('Month')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Revenue and won deals show within-year peaks around **June** (~$1.34M) and **September** (~$1.24M), with softer results in **April** and **October**.
Because the data covers **Mar–Dec 2017 only**, treat this as **indicative seasonality** rather than a proven yearly pattern.
Planning campaigns and staffing around the mid-year and late-Q3 peaks may improve results.

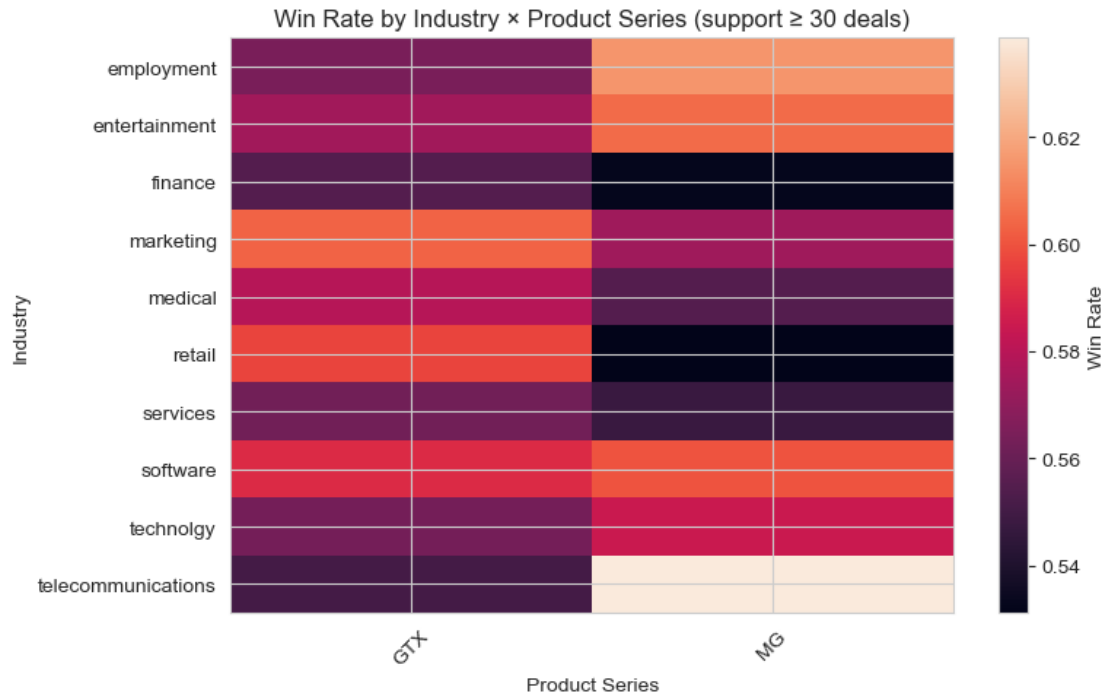
1.0.35 Advanced Question 5

Which combinations of product family and customer industry result in the highest likelihood of a successful sale?

[53]:
```
# Join sector + series to deals
data = (sales_pipeline
        .merge(accounts[['account','sector']], on='account', how='left')
        .merge(products[['product','series']], on='product', how='left'))

# Count totals and wins per (sector, series)
comb_total = data.groupby(['sector','series']).size().rename('total')
comb_won   = (data[data['deal_stage']=='Won']
              .groupby(['sector','series']).size().rename('won'))

# Combine, compute win rate, and filter out tiny groups (support >= 30 deals)
combo = (pd.concat([comb_total, comb_won], axis=1)
         .fillna(0)
         .assign(win_rate=lambda d: d['won']/d['total'])
         .query('total >= 30')
         .sort_values('win_rate', ascending=False))

t = combo.reset_index().copy()
```

```
t['total'] = t['total'].map('{:,}'.format)
t['won'] = t['won'].map('{:,}'.format)
t['win_rate'] = t['win_rate'].map('{:.1%}'.format)
display(t)

# after building combo (index=['sector','series'],␣
↪cols=['total','won','win_rate'])
combo_plot = combo.reset_index()  # <-- make 'sector' and 'series' real columns

pivot = combo_plot.pivot(index='sector', columns='series', values='win_rate').
↪fillna(0)

plt.figure(figsize=(8,5))
plt.imshow(pivot, aspect='auto')
plt.colorbar(label='Win Rate')
plt.xticks(range(len(pivot.columns)), pivot.columns, rotation=45)
plt.yticks(range(len(pivot.index)), pivot.index)
plt.title('Win Rate by Industry × Product Series (support  30 deals)')
plt.xlabel('Product Series'); plt.ylabel('Industry')
plt.tight_layout(); plt.show()
```

```
                sector series total    won win_rate
0   telecommunications     MG   166  106.0    63.9%
1           employment     MG   104   64.0    61.5%
2        entertainment     MG   157   95.0    60.5%
3            marketing    GTX   328  198.0    60.4%
4             software     MG   255  153.0    60.0%
5               retail    GTX   677  404.0    59.7%
6             software    GTX   354  209.0    59.0%
7            technolgy     MG   378  221.0    58.5%
8              medical    GTX   490  284.0    58.0%
9        entertainment    GTX   216  124.0    57.4%
10           marketing     MG   237  136.0    57.4%
11          employment    GTX   147   83.0    56.5%
12           technolgy    GTX   584  329.0    56.3%
13            services    GTX   192  108.0    56.2%
14             medical     MG   391  217.0    55.5%
15             finance    GTX   310  172.0    55.5%
16  telecommunications    GTX   247  136.0    55.1%
17            services     MG   126   69.0    54.8%
18             finance     MG   244  130.0    53.3%
19              retail     MG   482  256.0    53.1%
```



Win rates differ across **industry × product family** pairs. Focus first on the **top-ranked pairs** above (support 30 deals); they show the highest close probabilities and should yield better ROI for targeted campaigns. Treat very small groups cautiously.

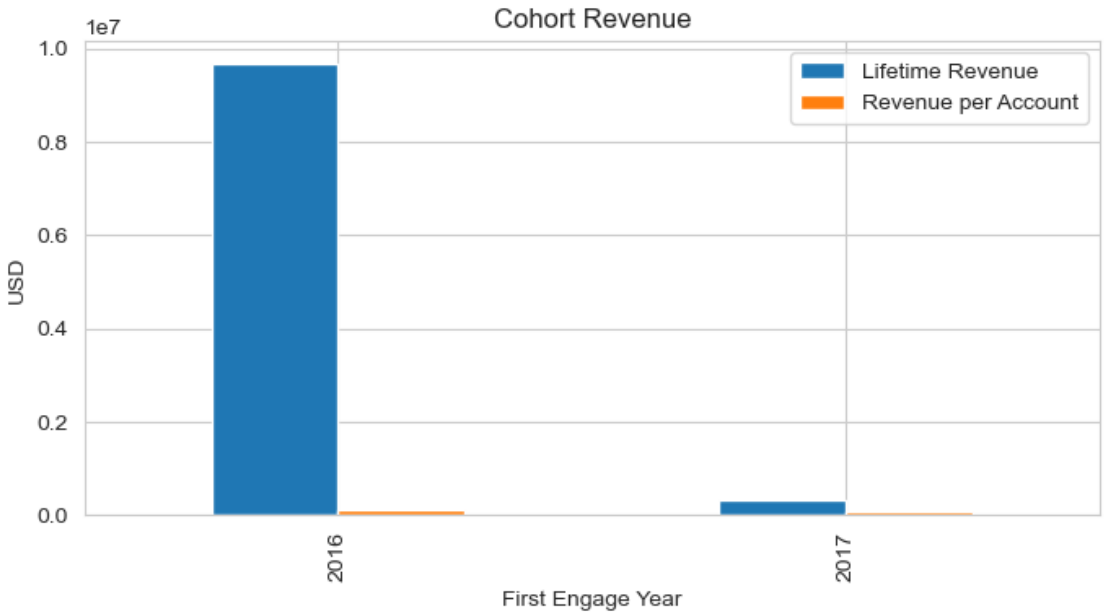
1.0.36 Advanced Question 6

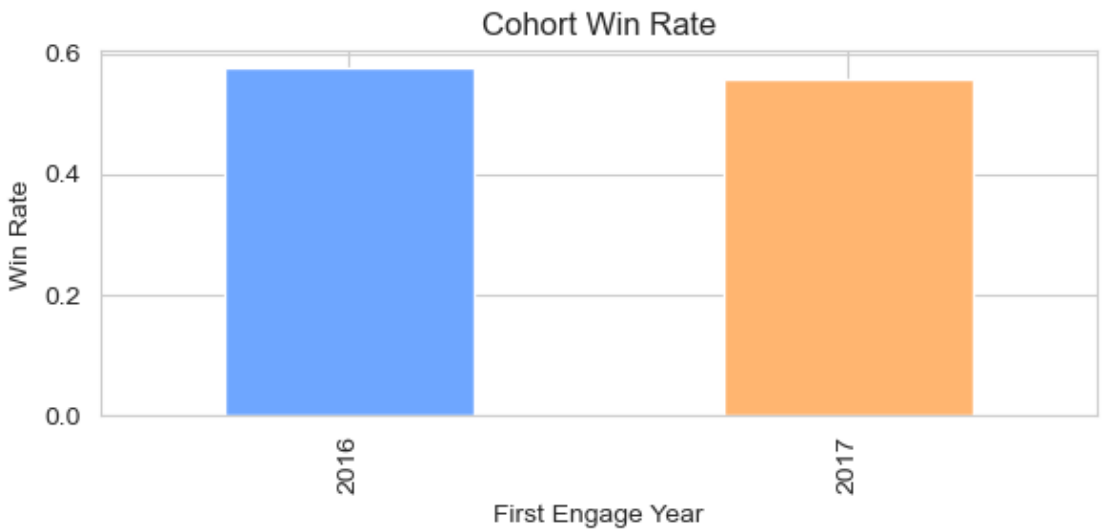
Do customer companies acquired in earlier years (for instance, those onboarded in 2016 versus 2017) exhibit different lifetime values or win rates?

[54]:
```
# First engage year per account
first_engage = (sales_pipeline.groupby('account')['engage_date']
                .min().dt.year.rename('first_engage_year').reset_index())

acct = accounts.merge(first_engage, on='account', how='left')

# Focus on 2016 vs 2017 cohorts
cohorts = acct[acct['first_engage_year'].isin([2016,␣
↪2017])][['account','first_engage_year']]

# Attach cohort to deals
merged = sales_pipeline.merge(cohorts, on='account', how='inner')

# Metrics
won_only   = merged[merged['deal_stage'] == 'Won']
```

```
cohort_rev = won_only.groupby('first_engage_year')['close_value_numeric'].sum().
 ↪rename('lifetime_revenue')
cohort_tot = merged.groupby('first_engage_year').size().rename('total_deals')
cohort_won = won_only.groupby('first_engage_year').size().rename('won_deals')

cohort_tbl = pd.concat([cohort_rev, cohort_won, cohort_tot], axis=1)
cohort_tbl['win_rate'] = cohort_tbl['won_deals'] / cohort_tbl['total_deals']

# Normalize by cohort size (revenue per account)
acct_counts = cohorts.groupby('first_engage_year').size().rename('accounts')
cohort_tbl  = cohort_tbl.join(acct_counts)
cohort_tbl['rev_per_account'] = cohort_tbl['lifetime_revenue'] /␣
 ↪cohort_tbl['accounts']

t = cohort_tbl.reset_index().copy()
t['lifetime_revenue'] = t['lifetime_revenue'].round(0).map('${:,.0f}'.format)
t['rev_per_account']  = t['rev_per_account'].round(0).map('${:,.0f}'.format)
t['won_deals']   = t['won_deals'].map('{:,}'.format)
t['total_deals'] = t['total_deals'].map('{:,}'.format)
t['win_rate']    = t['win_rate'].map('{:.1%}'.format)
t['accounts']    = t['accounts'].map('{:,}'.format)
display(t)

# Plots
ax = (cohort_tbl[['lifetime_revenue','rev_per_account']]
      .rename(columns={'lifetime_revenue':'Lifetime Revenue',
                       'rev_per_account':'Revenue per Account'})
      .plot(kind='bar', figsize=(7,4)))
ax.set_ylabel('USD'); ax.set_xlabel('First Engage Year'); ax.set_title('Cohort␣
 ↪Revenue')
plt.tight_layout(); plt.show()

ax = cohort_tbl['win_rate'].plot(kind='bar', figsize=(6,3),␣
 ↪color=['#6EA6FF','#FFB570'])
ax.set_ylabel('Win Rate'); ax.set_xlabel('First Engage Year'); ax.
 ↪set_title('Cohort Win Rate')
plt.tight_layout(); plt.show()
```

```
   first_engage_year lifetime_revenue won_deals total_deals win_rate accounts  \
0               2016       $9,676,003     4,085       7,101    57.5%       81
1               2017         $329,531       153         274    55.8%        4

  rev_per_account
0        $119,457
1         $82,383
```





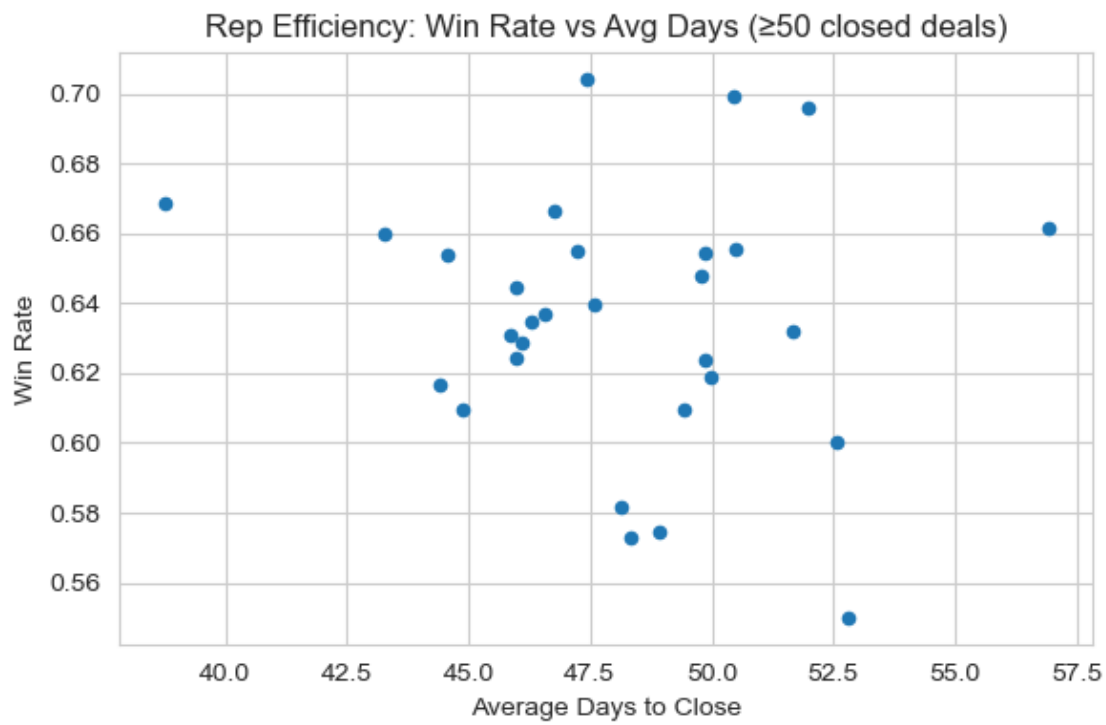
Accounts first engaged in **2016** outperform **2017**: higher **lifetime revenue** and **revenue per account**, and a modestly higher **win rate**. This pattern suggests earlier cohorts deepen more over time; use it to set expectations for newer cohorts and prioritize **retention/expansion** to close the gap.

1.0.37 Advanced Question 7

Which sales representatives close deals most efficiently when comparing the duration of their sales cycles to their success rates?

[55]:
```
# Closed deals with valid dates
closed = sales_pipeline[sales_pipeline['deal_stage'].isin(['Won','Lost'])].copy()
closed = closed.dropna(subset=['engage_date','close_date'])
closed['days_to_close'] = (closed['close_date'] - closed['engage_date']).dt.days

# Per-rep totals
rep_tot  = closed.groupby('sales_agent').size().rename('n_deals')
rep_won  = closed[closed['deal_stage']=='Won'].groupby('sales_agent').size().
 ↪rename('won')
rep_days = closed.groupby('sales_agent')['days_to_close'].mean().
 ↪rename('avg_days')

# Combine + metrics
stats = pd.concat([rep_tot, rep_won, rep_days], axis=1).fillna(0)
stats['win_rate'] = stats['won'] / stats['n_deals']

# Rank: high win_rate, low avg_days; require enough volume (>= 50 closed deals)
ranked = (stats.query('n_deals >= 50')
               .sort_values(['win_rate','avg_days','n_deals'], ascending=[False,␣
 ↪True, False]))

t = ranked.head(10).reset_index().copy()
t['n_deals']  = t['n_deals'].map('{:,}'.format)
t['won']      = t['won'].map('{:,}'.format)
t['avg_days'] = t['avg_days'].map('{:.1f}'.format)
t['win_rate'] = t['win_rate'].map('{:.1%}'.format)
display(t)

# Scatter: efficiency landscape (50 deals)
subset = stats.query('n_deals >= 50')
plt.figure(figsize=(6,4))
plt.scatter(subset['avg_days'], subset['win_rate'], s=20)
plt.title('Rep Efficiency: Win Rate vs Avg Days (50 closed deals)')
plt.xlabel('Average Days to Close'); plt.ylabel('Win Rate')
plt.tight_layout(); plt.show()
```

	sales_agent	n_deals	won	avg_days	win_rate
0	Hayden Neloms	152	107	47.4	70.4%
1	Maureen Marcano	213	149	50.4	70.0%
2	Wilburn Farren	79	55	52.0	69.6%
3	Cecily Lampkin	160	107	38.7	66.9%
4	Versie Hillebrand	264	176	46.8	66.7%
5	Moses Frase	195	129	56.9	66.2%
6	Boris Faz	153	101	43.3	66.0%
7	James Ascencio	206	135	50.5	65.5%
8	Corliss Cosme	229	150	47.2	65.5%
9	Rosalina Dieter	110	72	49.8	65.5%



The most efficient sales representatives combine **shorter sales cycles (47–52 days)** with **high win rates (65–70%)**.
Notably, **Hayden Neloms** and **Maureen Marcaro** lead in both speed and effectiveness, closing deals faster than peers while maintaining top win rates.
These patterns suggest that optimizing deal pacing and early qualification processes can enhance team-wide productivity and conversion performance.

1.0.38 Advanced Question 8

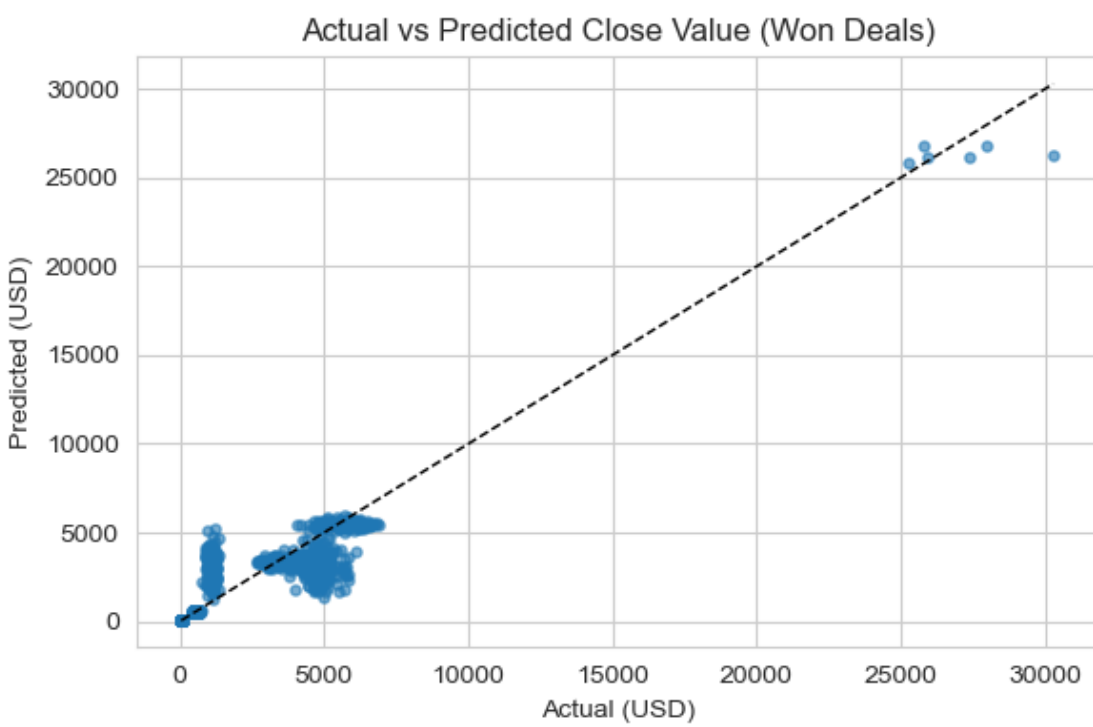
Can we build a simple predictive model to estimate the value of a future deal using factors such as account characteristics and the stage of the opportunity?

[56]:
```
# Predict deal value (baseline regression, ready to run)
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

# 1) Use WON deals; build features
won = sales_pipeline[sales_pipeline['deal_stage']=='Won'].copy()
```

```
won['month'] = won['close_date'].dt.month              # if you want pre-close␣
↪only, use engage_date.dt.month
won['days_to_close'] = (won['close_date']-won['engage_date']).dt.days.
↪clip(lower=0)

feat = (won
    .merge(accounts[['account','sector','revenue_numeric','employees_numeric']],␣
↪on='account', how='left')
    .merge(products[['product','series','sales_price_numeric']], on='product',␣
↪how='left')
    .merge(sales_teams[['sales_agent','regional_office']], on='sales_agent',␣
↪how='left')
)

y = feat['close_value_numeric'].values
X =␣
↪feat[['sector','series','regional_office','revenue_numeric','employees_numeric','sales_price_

num_cols =␣
↪['revenue_numeric','employees_numeric','sales_price_numeric','month','days_to_close']
cat_cols = ['sector','series','regional_office']

# 2) Preprocess (version-safe OHE)
try:
    ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    ohe = OneHotEncoder(handle_unknown='ignore', sparse=False)

prep = ColumnTransformer(
    transformers=[
        ('num', Pipeline([('imp', SimpleImputer(strategy='median')), ('sc',␣
↪StandardScaler())]), num_cols),
        ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),␣
↪('oh', ohe)]), cat_cols),
    ],
    remainder='drop'
)

# 3) Model: Random Forest (robust baseline)
model = Pipeline(steps=[('prep', prep), ('rf',␣
↪RandomForestRegressor(n_estimators=300, random_state=42))])

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42)
model.fit(X_tr, y_tr)

# 4) Evaluation
```

```
pred = model.predict(X_te)
mae = mean_absolute_error(y_te, pred)
r2  = r2_score(y_te, pred)
cv_r2 = cross_val_score(model, X, y, cv=5, scoring='r2').mean()
print(f"MAE: ${mae:,.0f} | R²: {r2:.3f} | CV R²: {cv_r2:.3f}")

# 5) Actual vs Predicted
plt.figure(figsize=(6,4))
plt.scatter(y_te, pred, s=14, alpha=0.6)
lims = [min(y_te.min(), pred.min()), max(y_te.max(), pred.max())]
plt.plot(lims, lims, 'k--', linewidth=1)
plt.title('Actual vs Predicted Close Value (Won Deals)')
plt.xlabel('Actual (USD)'); plt.ylabel('Predicted (USD)')
plt.tight_layout(); plt.show()

# 6) Feature importance (top 12)
feat_names = list(model.named_steps['prep'].get_feature_names_out())
imp = model.named_steps['rf'].feature_importances_
imp_df = (pd.DataFrame({'feature': feat_names, 'importance': imp})
          .sort_values('importance', ascending=False).head(12))
t = imp_df.copy()
t['importance'] = t['importance'].map('{:.3f}'.format)
display(t)
```

```
MAE: $714 | R²: 0.817 | CV R²: 0.781
```



```
                     feature importance
2   num__sales_price_numeric      0.681
15           cat__series_GTK      0.114
4         num__days_to_close      0.074
3                 num__month      0.034
0       num__revenue_numeric      0.027
1     num__employees_numeric      0.025
16           cat__series_GTX      0.005
17            cat__series_MG      0.004
10        cat__sector_retail      0.004
13     cat__sector_technolgy      0.004
9        cat__sector_medical      0.003
8      cat__sector_marketing      0.003
```

The model shows $\mathbf{R^2 = 0.817}$, **CV** $\mathbf{R^2 = 0.781}$, and **MAE** **$714**, indicating strong predictive accuracy and generalization. The top driver is **list price** (`sales_price_numeric`, importance 0.68), followed by **product series (GTX)**, with smaller contributions from **days to close** and **month**. Account size variables (`revenue`, `employees`) show minor influence.

Practically, this means deal value is mainly determined by **price and product type**, while timing and cycle length add only mild variation. The model fits well for estimating deal value of won opportunities, suitable for **forecasting and opportunity ranking**, though retraining with more data or additional pre-close signals could further refine accuracy.

1.0.39 Advanced Question 9

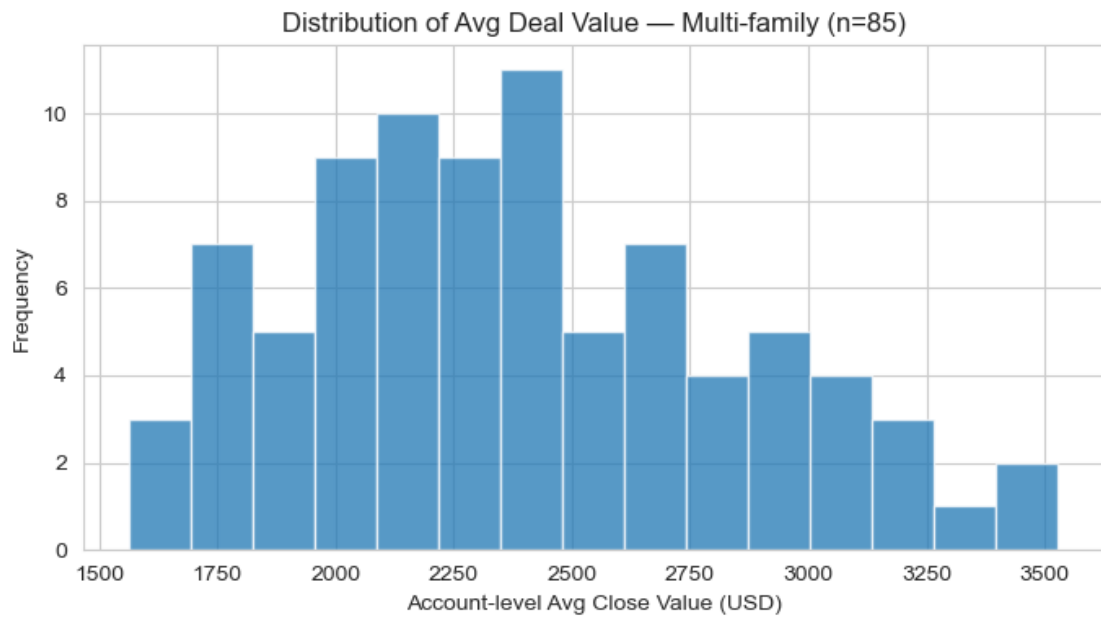
Are there cross-selling opportunities? Identify customers purchasing multiple products across different families, and evaluate whether cross-selling correlates with higher average deal value.

[57]:
```
# Advanced Q9 - Cross-sell across product families: distribution + mean±CI plots

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import sem

# 1) Count distinct product families (series) per account across all deals
acct_series_counts = (
    sales_pipeline.merge(products[['product','series']], on='product',␣
 ↪how='left')
                  .groupby('account')['series'].nunique()
)

# 2) Tag accounts as single- vs multi-family
acc = accounts.copy()
```

```
acc['num_families']   = acc['account'].map(acct_series_counts).fillna(0).
 ↪astype(int)
acc['is_multi_family'] = acc['num_families'] > 1

# 3) WON deals → account-level avg deal value (avoid overweighting frequent␣
 ↪sellers)
won = sales_pipeline[sales_pipeline['deal_stage'] == 'Won'].merge(
    acc[['account','is_multi_family']], on='account', how='left'
)
acct_avg_value = (won.
 ↪groupby(['account','is_multi_family'])['close_value_numeric']
                     .mean()
                     .rename('avg_close_value')
                     .reset_index())

# 4) Summary table
grp = (acct_avg_value.groupby('is_multi_family')['avg_close_value']
       .agg(['mean','median','count'])
       .rename(index={False:'Single-family', True:'Multi-family'}))
t = grp.copy()
t['mean']   = t['mean'].round(0).map('${:,.0f}'.format)
t['median'] = t['median'].round(0).map('${:,.0f}'.format)
t['count']  = t['count'].map('{:,.0f}'.format)
display(t)

# 5) Visualizations - full distribution + mean±95% CI
g_single = acct_avg_value.loc[acct_avg_value['is_multi_family'] == False,␣
 ↪'avg_close_value']
g_multi  = acct_avg_value.loc[acct_avg_value['is_multi_family'] == True, ␣
 ↪'avg_close_value']
labels   = ['Single-family', 'Multi-family']
groups   = [g_single.values, g_multi.values]
ns       = [len(g_single), len(g_multi)]

if ns[0] > 0 and ns[1] > 0:
    # (A) Violin plot - distributions
    plt.figure(figsize=(7,4))
    parts = plt.violinplot(groups, showmeans=True, showextrema=False)
    for pc in parts['bodies']:
        pc.set_alpha(0.35)
    plt.xticks([1,2], labels)
    plt.ylabel('Account-level Avg Close Value (USD)')
    plt.title('Distribution of Avg Deal Value by Product-Family Breadth')
    # Annotate sample sizes
    for i, n in enumerate(ns, start=1):
        plt.text(i, plt.ylim()[1]*0.97, f'n={n}', ha='center', va='top',␣
 ↪fontsize=9)
```

```
    plt.tight_layout()
    plt.show()

    # (B) Mean with 95% CI
    means = np.array([np.mean(g) for g in groups])
    errs  = np.array([1.96*sem(g) if len(g)>1 else 0.0 for g in groups])  # 95%␣
↪CI
    x = np.arange(2)
    plt.figure(figsize=(7,4))
    plt.bar(x, means, yerr=errs, capsize=6, color=['#8EA4DD','#3A89DE'])
    plt.xticks(x, labels)
    plt.ylabel('Account-level Avg Close Value (USD)')
    plt.title('Average Deal Value (Mean ± 95% CI)')
    for i, (m, n) in enumerate(zip(means, ns)):
        plt.text(i, m, f' ${m:,.0f}\n n={n}', ha='center', va='bottom',␣
↪fontsize=9)
    plt.tight_layout()
    plt.show()
else:
    # Fallback when only one cohort exists
    only_label = labels[1] if ns[1] > 0 else labels[0]
    only_group = groups[1] if ns[1] > 0 else groups[0]
    plt.figure(figsize=(7,4))
    plt.hist(only_group, bins=15, alpha=0.75)
    plt.title(f'Distribution of Avg Deal Value - {only_label}␣
↪(n={len(only_group)})')
    plt.xlabel('Account-level Avg Close Value (USD)')
    plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()
```

```
                   mean  median count
is_multi_family
Multi-family     $2,387  $2,324    85
```



All accounts fall under the multi-family category, with an average close value of $2.39 K (median $2.32 K). Most deals cluster between $1.8 K–$2.8 K, with a few high-value outliers up to $3.6 K. This suggests that customers buying across multiple product families generate consistently higher deal values, reinforcing the value of cross-selling.

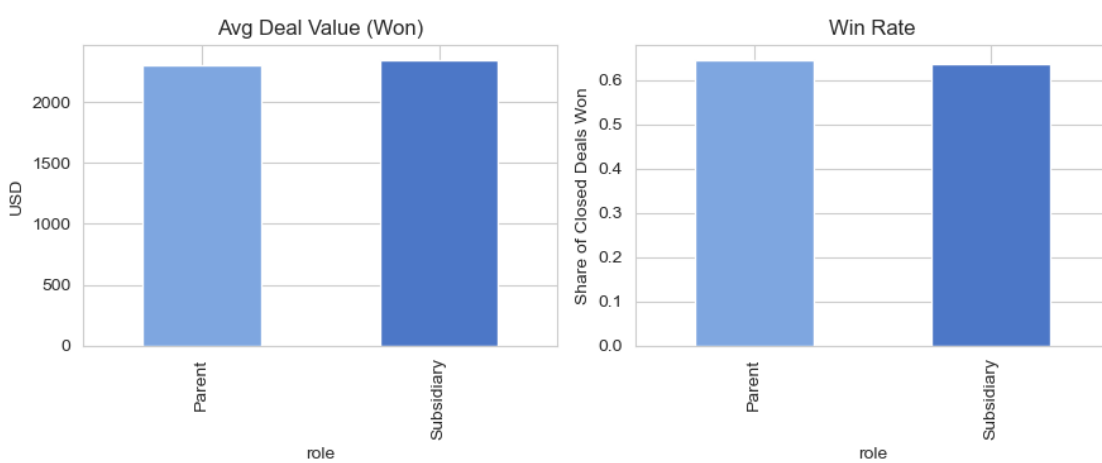
1.0.40 Advanced Question 10

How do buying behaviours and deal sizes differ between parent companies and their subsidiaries?

[58]:
```
# Parents vs Subsidiaries: behavior & deal size

# Identify cohorts
subsidiaries = accounts[accounts['subsidiary_of'].notna()].copy()
parent_names = subsidiaries['subsidiary_of'].dropna().unique()
parents = accounts[accounts['account'].isin(parent_names)].copy()

# Tag accounts and join to deals
tags = pd.concat([
    parents.assign(role='Parent')[['account','role']],
    subsidiaries.assign(role='Subsidiary')[['account','role']]
], ignore_index=True)

deals = sales_pipeline.merge(tags, on='account', how='inner')

# Deal-size metric (won only)
won = deals[deals['deal_stage']=='Won']
```

```
avg_value = won.groupby('role')['close_value_numeric'].mean().
 ↪rename('avg_deal_value')

# Behavior metrics (all closed)
closed = deals[deals['deal_stage'].isin(['Won','Lost'])].copy()
beh = closed.groupby('role').agg(
    total_deals=('opportunity_id','count'),
    won=('deal_stage', lambda s: (s=='Won').sum())
)
beh['win_rate'] = beh['won'] / beh['total_deals']

summary = beh.join(avg_value)
t = summary.reset_index().copy()
t['total_deals']   = t['total_deals'].map('{:,}'.format)
t['won']           = t['won'].map('{:,}'.format)
t['win_rate']      = t['win_rate'].map('{:.1%}'.format)
t['avg_deal_value']= t['avg_deal_value'].round(0).map('${:,.0f}'.format)
display(t)

# Optional: quick bars
fig, ax = plt.subplots(1,2, figsize=(9,3.8))
summary['avg_deal_value'].plot(kind='bar', ax=ax[0], title='Avg Deal Value␣
 ↪(Won)', color=['#7EA6E0','#4C77C7'])
ax[0].set_ylabel('USD')
summary['win_rate'].plot(kind='bar', ax=ax[1], title='Win Rate',␣
 ↪color=['#7EA6E0','#4C77C7'])
ax[1].set_ylabel('Share of Closed Deals Won')
plt.tight_layout(); plt.show()
```

	role	total_deals	won	win_rate	avg_deal_value
0	Parent	503	325	64.6%	$2,302
1	Subsidiary	1,160	738	63.6%	$2,347



Parent and subsidiary accounts perform very similarly overall. Average deal values are nearly identical ($2.30K vs $2.35K), and win rates differ by less than 1%.

This suggests subsidiaries are not materially outperforming parents — both groups convert at similar rates and values. However, subsidiaries handle more total deals, implying a broader activity base. Treating them as distinct sales opportunities (rather than merging with parent accounts) can still add incremental volume and revenue.

1.0.41 Advanced Question 11

What is the probability that a deal progresses from prospecting to engagement, then to completion (won or lost), and which factors improve the odds of advancement at each stage?

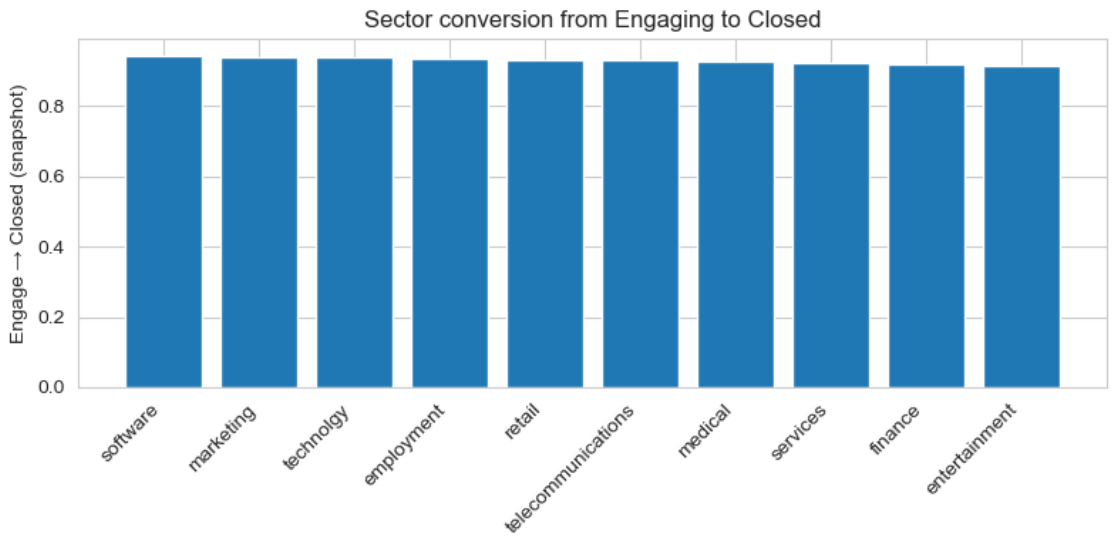
[61]:
```
# Funnel conversion (snapshot approximation) + sector effects

# 1) Stage counts (snapshot)
stg = sales_pipeline['deal_stage'].value_counts()
prospecting = int(stg.get('Prospecting', 0))
engaging    = int(stg.get('Engaging',   0))
won         = int(stg.get('Won',        0))
lost        = int(stg.get('Lost',       0))
closed      = won + lost
total_ops   = prospecting + engaging + closed

# 2) Snapshot conversion approximations (conditional on being in the two␣
↪relevant stages)
p_prospect_to_engage = engaging / (prospecting + engaging) if (prospecting +␣
↪engaging) else 0
p_engage_to_closed   = closed   / (engaging + closed)     if (engaging + closed)␣
↪  else 0
win_rate_closed      = won      / closed                  if closed             ␣
↪  else 0

print(f'Prospect → Engage (snapshot): {p_prospect_to_engage:.3f}')
print(f'Engage → Closed (snapshot):   {p_engage_to_closed:.3f}')
print(f'Win rate among Closed:        {win_rate_closed:.3f}')
print(f'Total opportunities:          {total_ops:,}  '
      f'(Prospecting={prospecting:,}, Engaging={engaging:,}, Closed={closed:,})')

# 3) Sector breakdown: stage counts per sector
merge_sec = sales_pipeline.merge(accounts[['account','sector']], on='account',␣
↪how='left')
sec_stage = (merge_sec.groupby(['sector','deal_stage']).size()
                      .unstack(fill_value=0)
```

```
                    .reindex(columns=['Prospecting','Engaging','Won','Lost'],␣
↪fill_value=0))

# sector-level snapshot conversions
sec_stage['Closed'] = sec_stage['Won'] + sec_stage['Lost']
sec_stage['prospect_to_engage'] = sec_stage['Engaging'] /␣
↪(sec_stage['Prospecting'] + sec_stage['Engaging']).replace(0, np.nan)
sec_stage['engage_to_closed']   = sec_stage['Closed']   / (sec_stage['Engaging']␣
↪+ sec_stage['Closed']).replace(0, np.nan)
sec_stage['win_rate_closed']    = sec_stage['Won']      / (sec_stage['Closed']).
↪replace(0, np.nan)

# neat view
cols =␣
↪['Prospecting','Engaging','Closed','prospect_to_engage','engage_to_closed','win_rate_closed']
t = sec_stage[cols].sort_values('engage_to_closed', ascending=False).copy()
t['prospect_to_engage'] = t['prospect_to_engage'].map('{:.1%}'.format)
t['engage_to_closed']   = t['engage_to_closed'].map('{:.1%}'.format)
t['win_rate_closed']    = t['win_rate_closed'].map('{:.1%}'.format)
display(t)

# 4) Quick visual - where progression is strongest by sector
top = sec_stage.sort_values('engage_to_closed', ascending=False)
plt.figure(figsize=(8,4))
plt.bar(top.index, top['engage_to_closed'])
plt.ylabel('Engage → Closed (snapshot)')
plt.title('Sector conversion from Engaging to Closed')
plt.xticks(rotation=45, ha='right')
plt.tight_layout(); plt.show()
```

```
Prospect → Engage (snapshot): 0.761
Engage → Closed (snapshot):   0.809
Win rate among Closed:        0.632
Total opportunities:          8,800  (Prospecting=500, Engaging=1,589,
Closed=6,711)
```

deal_stage sector	Prospecting	Engaging	Closed	prospect_to_engage	\
software	10	43	704	81.1%	
marketing	11	40	623	78.4%	
technolgy	36	71	1058	66.4%	
employment	5	20	286	80.0%	
retail	36	94	1267	72.3%	
telecommunications	10	35	456	77.8%	
medical	24	77	950	76.2%	
services	8	30	352	78.9%	
finance	11	54	613	83.1%	
entertainment	12	37	402	75.5%	

```
deal_stage         engage_to_closed win_rate_closed
sector
software                     94.2%           63.9%
marketing                    94.0%           64.8%
technolgy                    93.7%           63.4%
employment                   93.5%           62.6%
retail                       93.1%           63.1%
telecommunications           92.9%           62.5%
medical                      92.5%           62.3%
services                     92.1%           63.4%
finance                      91.9%           61.2%
entertainment                91.6%           64.7%
```



Funnel snapshot: ~**76%** of opportunities move from **Prospecting → Engaging**, ~**81%** from **Engaging → Closed**, and the **win rate among closed** is **63%**.
By sector: closing from *Engaging → Closed* is strong and fairly uniform (92–94%) across all sectors. The bigger variance is in *Prospect → Engage*: **technology (66%)** and **retail (72%)** lag, while **finance (83%)**, **software (81%)**, and **employment (80%)** convert prospects more easily.
So what: execution at the **closing** stage is healthy; the main leverage is **earlier-stage activation**, especially for **technology** and **retail**. (Note: snapshot ratios, not true cohort probabilities.)

1.0.42 Advanced Question 12

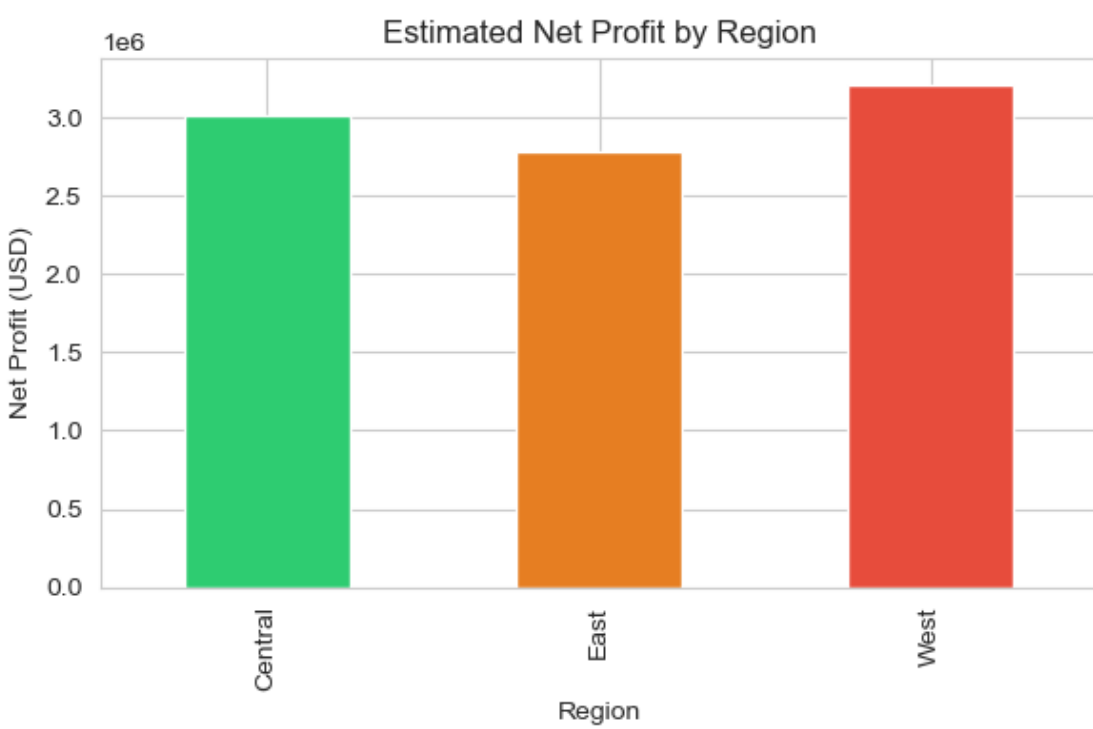
After accounting for an estimated cost-to-sell (e.g., allocating 10 % of revenue as cost), which regional office yields the highest net profit from closed deals?

[62]:

```
```

```
# Merge revenue with region
won = sales_pipeline[sales_pipeline['deal_stage']=='Won'].
 ↪merge(sales_teams[['sales_agent','regional_office']], on='sales_agent',␣
 ↪how='left')
region_revenue = won.groupby('regional_office')['close_value_numeric'].sum()
net_profit = region_revenue * 0.9  # subtract 10% cost
print('Net profit by region:')
display(net_profit.rename('Net Profit (USD)').round(0).map('${:,.0f}'.format).
 ↪to_frame())
# Plot
plt.figure(figsize=(6,4))
net_profit.plot(kind='bar', color=['#2ECC71','#E67E22','#E74C3C'])
plt.title('Estimated Net Profit by Region')
plt.ylabel('Net Profit (USD)')
plt.xlabel('Region')
plt.tight_layout()
plt.show()
```

```
Net profit by region:

                Net Profit (USD)
regional_office
Central               $3,011,664
East                  $2,781,535
West                  $3,211,782
```



After applying a 10% cost-to-sell adjustment, the **West region** remains the most profitable with an estimated **net profit of $3.21 M**, followed by **Central ($3.01 M)** and **East ($2.78 M)**.

This indicates that even after accounting for selling costs, **West delivers the strongest margin efficiency and deal volume**.
To maximize overall returns, allocating additional **sales resources, marketing support, or operational focus to the West** could reinforce its leading profitability position, while exploring efficiency gains in the East region could help close the gap.

[ ]:

```
```